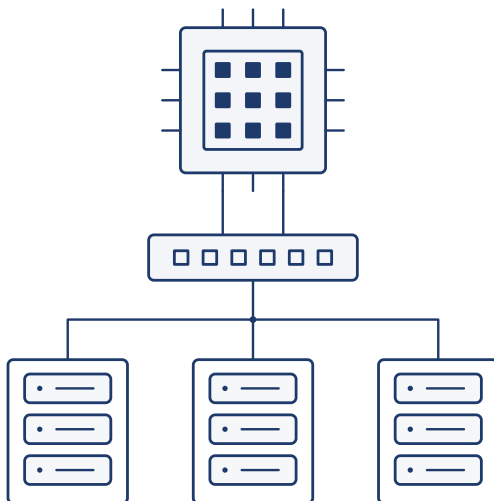


深入理解 AI Infra

量化分析与系统设计



李博杰 著

目录

第 12 章 端边云协同	1
12.1 从模型运行时间到完整交互时间	1
12.2 跨设备分工	8
12.3 实际传输中的额外开销与等待	13
12.4 无线变化下的交互与恢复	17
12.5 Queqiao 性能分析	20
12.6 从端边云分工到完整部署	23
练习	29
本章小结	30

第 12 章 端边云协同

一张图片交给云端精修，模型运行 0.3 秒，用户却等待 12.8 秒。将模型加速十倍，只能省下 0.27 秒；把输入压缩一半，却能省下近 6 秒。决定模型放在哪里时，需要比较整个任务的完成时间。远端设备可以加快模型计算，但调用远端服务也会增加传输和等待时间。

前面的章节介绍了模型服务、资源调度和任务运行环境。本章讨论如何把这些服务和环境部署到不同设备上。**端侧**采集输入并与用户交互；**边缘**是在附近提供计算的工作站或服务节点；**云端**提供远程资源池。终端可以在本地执行，也可以调用附近工作站或云端服务，还可以把模型的不同阶段分配给不同设备。

我们先用图片精修建立时间模型，再用语音解释流水线中的先后依赖，最后为一个截图 Agent（根据截图判断下一步并调用工具执行操作的程序）选择部署方案。图片精修需要计算完整成片何时返回，语音需要分析音频能否连续播放，Agent 则要累计每轮通信与执行的开销。三类任务共用一套分析方法：先计算处理和传输各需多久，再确定哪些步骤必须等待，最后加入连接建立、排队和故障恢复等实际开销。

12.1 从模型运行时间到完整交互时间

12.1.1 图片精修

图片精修需要保留阴影、高光和白平衡等信息。为后期编辑保留更多原始信息，通常也意味着需要传输更大的文件。本例上传一张 30 MB 原图，处理后返回一张 5 MB 成片，以用户收到完整成片为任务结束。¹

先沿一张图片的路径看数据在何处停留。本例的具体速率和处理时间标在图中，下面再把这些量写成通用公式。

建立模型时，先区分**各步骤所需时间**与**任务完成时间**。文件大小除以发送速率，得到发送所需时间；模型的计算时间则由计算量和处理器的执行速度决定。各步骤依次执行时，总时间等于各步骤耗时之和；部分步骤同时执行时，则要根据它们的先后依赖，计算最后一步何时结束。

本例将 RTT 简化为往返传播所需的时间，排队和协议处理的耗时在 12.3 节单独讨论。连接已经建立，服务器收齐原图后开始处理，生成完整成片后再向用户发送，上下行以恒定速率传输有效载荷。设输入为 S_u bytes，输出为 S_d bytes，上下行速率为 B_u, B_d bytes/s，往返时间（round-trip time, RTT）为 R ，处理时间为 T_c 。串行完成时间为

$$T_{\text{serial}} = \frac{S_u}{B_u} + R + T_c + \frac{S_d}{B_d}.$$

上传的最后一个字节离开发送端后，还要传播到服务器；成片返回也要传播一次。两次单向传播合计为 R 。传播时间由路径决定，发送时间由文件大小与速率决定，因此缩小文件只改变发送项。这里的 B_u, B_d 表示有效载荷的传输速率，连接启动和反馈等待将在 12.3 中单独加入。

例：图片精修的上传时间如何限制模型加速收益？ 上行 20 Mbit/s，下行 100 Mbit/s， $R = 0.1$ s， $T_c = 0.3$ s。本章 MB 与 Mbit 使用十进制；MiB 为 2^{20} bytes。代入得

¹RAW 图片精修案例与条件预算。RAW / DNG 与信息保留由归档的华为专业模式及 Adobe 材料支持；本章 30 MB / 5 MB / 0.3 秒为教学输入，不是历史服务规格。

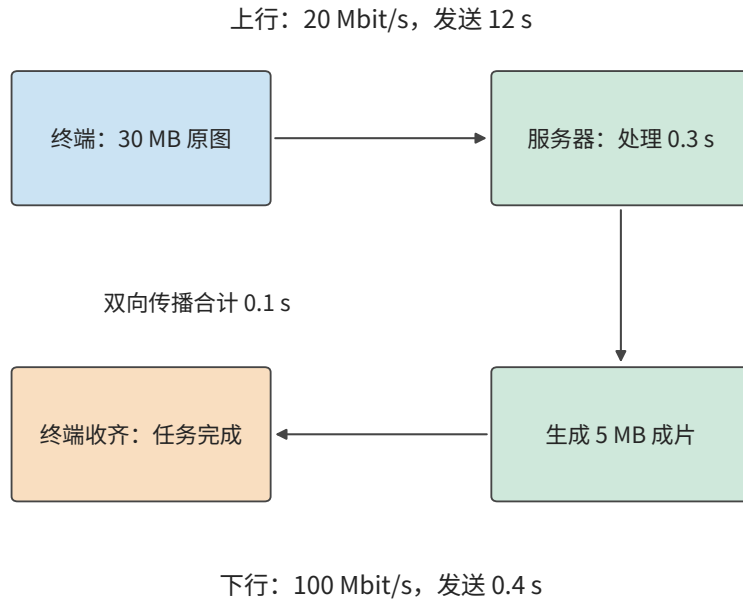


图 12-1：原图经上行到服务器，服务器收齐后处理，再经下行返回完整成片。实线箭头表示数据与处理顺序；连接已建立，双向传播合计 0.1 s。

$$T_{\text{serial}} = \frac{30 \times 8}{20} + 0.1 + 0.3 + \frac{5 \times 8}{100} = 12.8 \text{ s.}$$

上传占 12 秒，约为总时间的 94%。将处理阶段加速十倍，完成时间缩短至 12.53 秒，只节省 0.27 秒。即使处理瞬间完成，上传、传播和下载仍需 12.5 秒。这正是 Amdahl 定律所描述的限制：处理阶段只占总时间的约 2.3%，即使消除这部分耗时，总时间也只能缩短约 2.3%。

把上行速率提高至 100 Mbit/s，上传时间降至 2.4 秒，整个任务只需 3.2 秒，速度提高到原来的四倍。另一种办法是保持上行速率不变，压缩输入。假设无损压缩将输入减半，额外的编解码共需 0.15 秒，则发送只需 6 秒，完整任务约为 7.0 秒。压缩之所以值得，是因为用 0.15 秒计算替换了 6 秒传输。

设压缩后的输入大小为 S'_u ，额外的编解码时间为 T_e ，压缩省下的总时间为

$$\Delta T = \frac{S_u - S'_u}{B_u} - T_e.$$

令这个式子中的收益为零，就能求出压缩恰好不再省时的上行速率。上行速率为 20 Mbit/s 时，少传 15 MB 省 6 秒；为 100 Mbit/s 时省 1.2 秒；达到 800 Mbit/s 时恰好只省 0.15 秒，收益降至零。同一种压缩方法的运算量没有改变，改变的是发送这些字节需要多少时间。图 12-2 显示了这种关系。

接下来考虑各步骤能否同时进行。假设原图分成三块后可以逐块独立处理，每块上传需 4 秒、处理需 0.1 秒；三块输出分别为 1、2、2 MB，回传依次需 0.08、0.16、0.16 秒。第一块上传后，其处理和回传可以与第二块上传重叠；第二块同理。最后一块仍要等到第 12 秒才发完，之后再经过处理、回传和传播，完整成片约在 12.4 秒返回用户。与原来的 12.8 秒相比，前两块的处理与回传在后续上传期间就已完成，不再额外增加等待；总计 12 秒的上传时间仍然需要。²

²分三块的两种计算分别为 12.8 s 与 12.36 s，正文以约 0.4 s 差额解释重叠收益；额外预览另增编码与传输，不等同完整成片。来源：整图

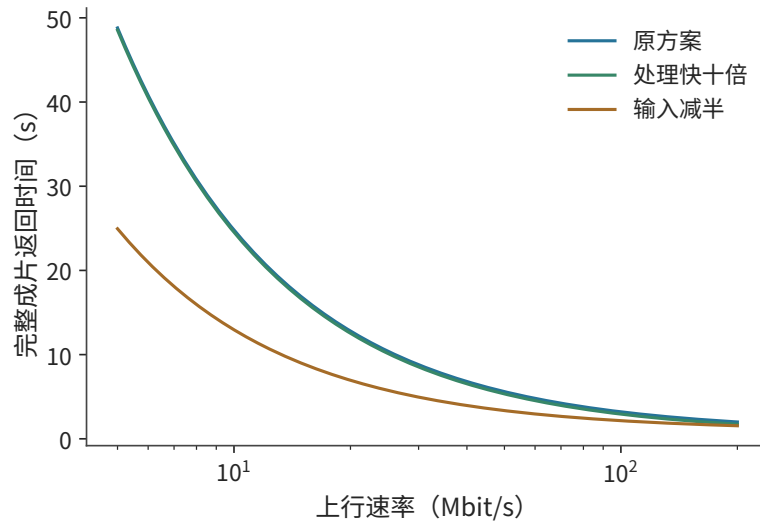


图 12-2: 压缩和计算加速对任务完成时间的影响随上行速率而变化。本例采用原图 30 MB、成片 5 MB、下行 100 Mbit/s、RTT 0.1 秒，原处理时间 0.3 秒；压缩方案将输入减半并增加 0.15 秒编解码。三种方案的成片质量相同，连接已建立且各阶段串行。

图 12-3 和图 12-4 用相同的时间轴比较两种安排：看前两块时，处理与回传都发生在后续上传期间；看最后一块时，上传结束后的等待仍然保留。

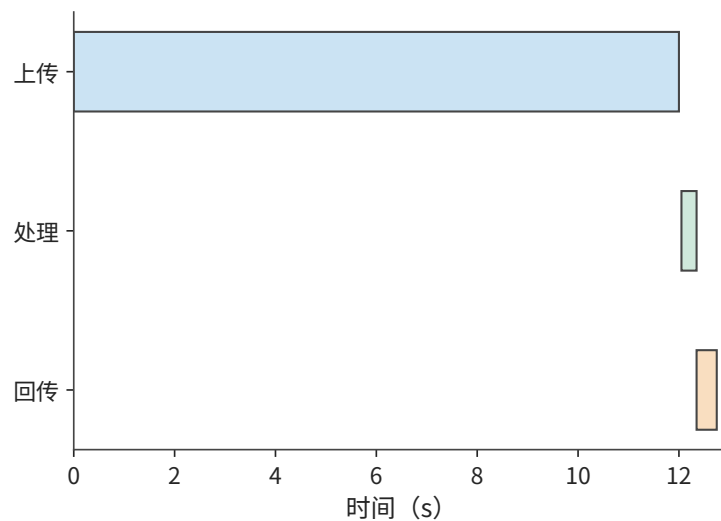


图 12-3: 整图串行：30 MB 全部上传并传播到服务器后才处理，随后回传 5 MB。上行 20 Mbit/s、下行 100 Mbit/s、单向传播 0.05 s，任务于 12.8 s 完成。

如果处理需要全局曝光或跨块信息，就要等整张图到齐后才能开始处理，即使分块发送，整个任务仍需 12.8 秒。接下来要分析步骤之间的依赖：计算量和数据量决定各步骤所需时间，依赖关系决定它们能否同时执行。实时语音服务会持续接收、处理和播放音频，更适合用流水线描述。

屏障、独立分块及额外预览。各自保留依赖与编码条件。

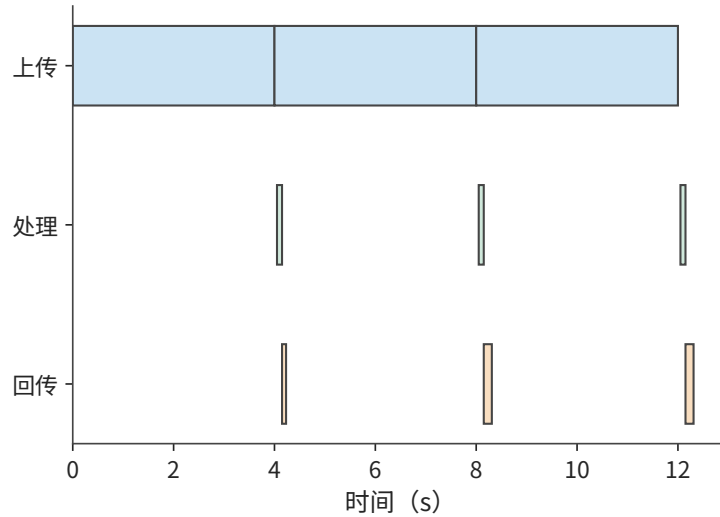


图 12-4: 三块各上传 4 s，每块到达后独立处理 0.1 s，随后回传 1、2、2 MB。前两块的处理和回传与后续上传重叠，完整成片于 12.36 s 返回。

12.1.2 实时语音

自动语音识别（ASR）将音频转为文本，文本转语音（TTS）将文本转为可播放音频。流式服务可以一边接收输入，一边产生结果。以 TTS 为例，用户可以在整段音频生成完之前开始收听。因此，除了整段合成何时结束，还需要知道首段何时开始播放，以及后续播放是否连续。

考虑一条输入块与输出块一一对应的音频处理流水线。采集端每 20 ms 产生一块音频，模型处理后将结果发出，播放器按顺序播放。第 i 块输入在 a_i 时刻就绪，模型处理耗时为 m_i ，发送耗时为 s_i ，单向传播为 d_i 。处理器逐块处理音频，网络接口逐块发送结果，则

$$c_i = \max(a_i, c_{i-1}) + m_i, \quad u_i = \max(c_i, u_{i-1}) + s_i, \quad r_i = u_i + d_i.$$

其中， c_i 、 u_i 、 r_i 分别表示处理完成、发送完成和到达的时刻。第一个最大值表示模型既要等输入，也要等上一块释放处理器；第二个最大值表示发送既要等计算，也要等出口空闲。这两个等待来源分别是数据依赖和资源竞争。

例：网络延迟抖动为何会耗尽音频播放缓冲？ 脉冲编码调制（PCM）逐次保存音频幅值采样；24 kHz 表示每秒 24000 次采样，单声道表示一条采样通道，16-bit 表示每个样本占 2 字节。使用 24 kHz、单声道、16-bit PCM，每块 20 ms、960 bytes。处理一块需 12 ms，发送需 1 ms，通常传播 5 ms。第一块在 20 ms 采集完，于 38 ms 到达；缓冲 40 ms 后在 78 ms 播放。第二块在 58 ms 到达，于 98 ms 接着播放。每块的处理时间比采集间隔短 8 ms，发送也只需 1 ms，因此处理和发送都能跟上采集速度。

现在把第三块的传播时间改为 50 ms，它在 123 ms 到达，比原定播放的 118 ms 晚 5 ms。第四块虽然在 98 ms 已经到达，播放器仍须先播第三块，因而后续播放一起推迟。³

令每块音频的时长为 τ ，初始缓冲为 J 。首块开始播放的时刻为 $p_0 = r_0 + J$ ，后续各块的开始时刻满足

³音频块时序计算，输入帧与输出音频块一一对应是该模型的简化；该简化用于说明逐块处理与按序播放的依赖。

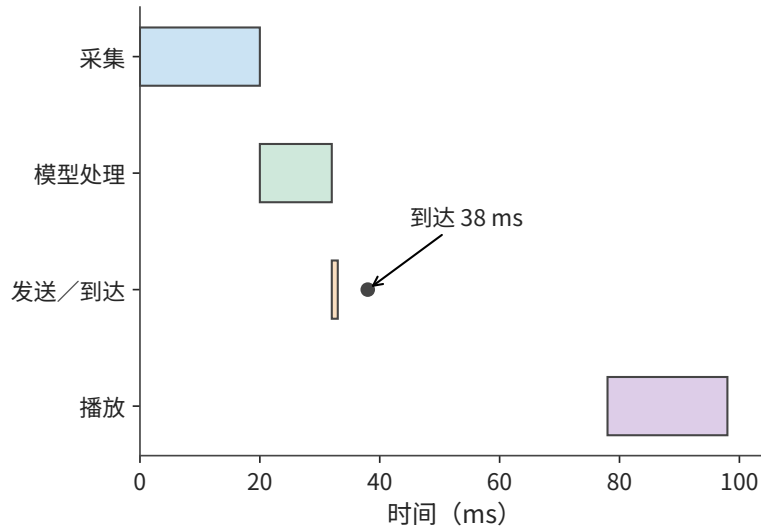


图 12-5：第一块音频从采集开始计时：20 ms 就绪，32 ms 处理完，33 ms 发完，38 ms 到达；初始缓冲 40 ms 后于 78 ms 播放。圆点标出到达，最后一行是播放设备的时钟。

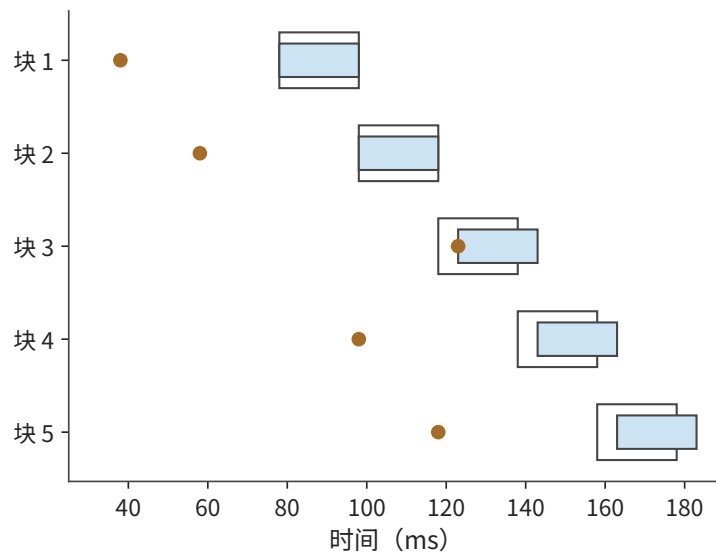


图 12-6：第三块晚到使后续各块的播放时间都推迟 5 ms。每块长 20 ms，模型处理 12 ms、发送 1 ms；通常传播 5 ms，第三块传播 50 ms，初始缓冲 40 ms。实线段表示实际播放，浅色轮廓表示原定播放区间，圆点表示到达；所有时间均从开始采集第一块起算。图为上述教学流水的计算结果，播放器等待缺块而不丢弃。

$$p_i = \max(r_i, p_{i-1} + \tau).$$

这里取最大值，是因为播放器既要等待“本块到达”，也要等待“上一块播完”。当第一项更大时，差额就是新增停顿。把初始缓冲从 40 ms 增至 45 ms，第三块的计划播放时刻也推迟到 123 ms，恰好吸收晚到的 5 ms；代价是首段同样晚开始 5 ms。增加初始缓冲会延长首次播放前的等待，但也能吸收更大的到达时间波动，使后续播放更连续。

长期传输速率不足会产生另一种停顿。为便于计算，取 16 kHz、单声道、16-bit PCM，播放所需的数据速率为 256 kbit/s，20 ms 对应 640 bytes。若网络长期只能以 130 kbit/s 传输音频，缓冲每秒净减少 126 kbit。初

始缓冲保存的 60 ms 音频包含 15,360 bits，约 $15360/(256000 - 130000) \approx 0.12$ 秒便会耗尽。

把尚未播放的数据画成随时间下降的曲线，就能看出短时抖动与长期速率不足的区别。图 12-7 中，增加缓冲只抬高起点，斜率没有改变。

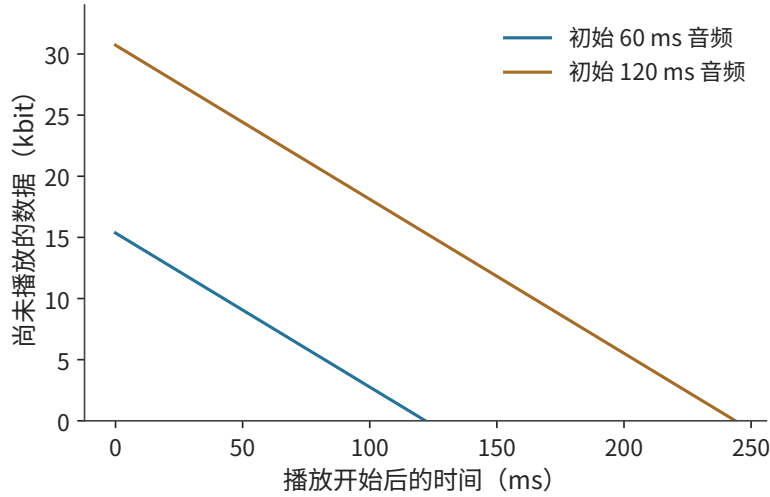


图 12-7：缓冲量等于初始数据量加累计接收量，再减累计播放量。PCM 播放需要 256 kbit/s，接收只有 130 kbit/s，两条直线的下降速率均为 126 kbit/s。初始缓冲分别包含 60 ms 和 120 ms 音频，约在 0.12 秒和 0.24 秒耗尽；图画到各自第一次耗尽为止。

增大缓冲只会按比例延后耗尽；提高传输速率或降低音频码率，才能让收到的数据足够播放。短时抖动可以靠缓冲吸收，长期传输速率不足则需要增加带宽或减少数据量。

12.1.3 Computer Use：通过界面操作完成任务

Computer Use（计算机界面操作）让程序像用户一样通过截图、点击和输入完成任务。它在观察界面、模型判断、执行操作和再次观察之间循环。它与音频流水的关键区别是：只有上一轮操作完成、界面更新后，才能取得下一张有效截图。模型还没决定点击哪里时，就无法取得点击后的界面。因此，一轮中的等待会在任务中重复累积。

图 12-8 的返回箭头说明了为什么这一任务难以像音频那样逐块重叠：后续工作必须等待操作产生新的界面。

以下以一个需要 30 轮操作的任务为例。使用原有服务时，每轮上传 0.8 MB 截图，上行 6.4 Mbit/s，上传需 1 秒；终端观测、操作与界面更新合计 0.3 秒，远端模型 2.0 秒，往返传播 0.2 秒。连接已建立，控制回复的发送时间取零。一轮为 $1 + 0.3 + 2.0 + 0.2 = 3.5$ 秒，30 轮共 105 秒。

例：截图压缩怎样改变完整任务？ 将截图压缩至 0.2 MB，额外编码需 30 ms，上传时间降为 0.25 秒，其余工作仍为 2.5 秒。于是

$$T'_{\text{round}} = 0.25 + 0.03 + 2.5 = 2.78 \text{ s.}$$

每轮省 0.72 秒，30 轮省 21.6 秒，任务从 105 秒降至约 83 秒。每轮都要传输截图，因此单轮不到一秒的改善累计起来，就能让整个任务提前二十多秒完成。

压缩还可能改变操作轮数。若文字变模糊使 Agent 多执行纠错操作，压缩后的总时间应写为 $2.78N'$ ，其中

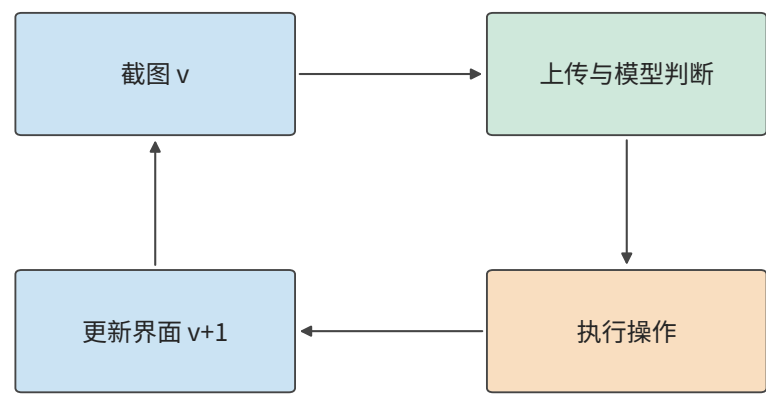


图 12-8：截图 Agent 的一轮执行。箭头表示先后依赖，方框宽度不表示耗时。底部返回箭头要经过操作执行与界面更新，才能取得下一轮截图。

N' 是实际所需轮数。由 $2.78N' < 105$ 得 $N' < 37.8$ ，仍能缩短任务时间的最大轮数为 37；38 轮约需 106 秒。压缩后的图像质量会影响纠错次数。这个任务最多允许增加七轮操作；超过这一数量，纠错时间就会抵消压缩省下的时间。

截图版本也直接影响轮数。假设模型读到版本 v ，随后界面变化，按旧截图作出的操作就可能出错，需要恢复界面并重新截图。把截图版本与动作编号绑定，让执行端在界面变化时重新截图，可以避免这类无效工作。⁴

12.6 将回到这个任务：前十轮已完成，剩余二十轮需要选择执行位置。届时保留截图大小、终端工作和原路径参数，引入更快的模型服务，比较切换服务后能缩短多少时间。

12.1.4 本地、附近设备与云端的初步比较

现在可以用同一套方法分析这三类任务。

分析层次	计算对象	已得到的判断
各步骤耗时	计算、发送和传播各需多久	原图上传占 12 秒，模型占 0.3 秒
执行依赖	哪些工作可以重叠，哪些必须等待	独立图片块可重叠，下一轮截图要等上一轮操作完成
实际执行	数据就绪后，计算和发送何时开始	音频即使平均传输速率足够，一次晚到仍可造成停顿

先用各步骤耗时与依赖关系判断远端执行能否更快。将整个任务交给远端服务时，省下的计算时间必须超过上传、下载、传播、排队和准备增加的时间。对于多轮任务，准备只需做一次，通信却每轮都会发生；对于连续播放的音频，还要保证数据及时到达。模型放在哪里，会同时改变计算、通信和准备时间。

最短传播时间可以帮助我们尽早排除距离过远的服务器。光在光纤中约以 2×10^8 m/s 传播，单向路径长 10000 km 时，往返传播至少需要约 100 ms。如果要求 50 ms 内收到远端应答，仅传播时间就已超期。使用更

⁴作者材料与技术判断。AOI 与 Latent Bridge 用于说明应观察哪些界面信息，以及设备之间应传递什么数据，本章未把其任务表现推广为通用 Agent 性能。

近的服务器，可以直接缩短这部分等待。

确定哪些设备来得及完成任务后，接下来要决定传什么。整项远程调用传图片，端侧编码传特征，会话迁移传状态；传输对象改变后，数据量和所需时间也随之改变。

12.2 跨设备分工

12.2.1 完整模型调用与分阶段执行

前面按图片大小估算上传时间，默认传送的是原始输入。将部分模型移到端侧后，这个默认条件就变了。把整个模型放到远端，需要上传输入并下载结果。把编码器（将图片或音频转换为模型使用的数值特征的子网络）留在端侧，则需要上传编码器的输出。这样，即使执行同一个任务，上传的也可能是图片或特征；两者的数据量取决于各自的编码方式。

对于截图 Agent，压缩图片中的纹理和重复区域可以用较少字节表示；视觉编码器却要把图片展开为每个视觉位置的通道值，以供语言模型读取。前者追求紧凑存储，后者服务于后续计算。在端侧运行编码器，既改变了执行编码的设备，也把上传对象从图片换成了特征。

比较两种方案：一种先上传图片，再在远端编码；另一种先在本地编码，再上传特征。两者后续的语言模型计算相同，计算总时间之差时，这部分正好抵消。因此，只需比较本地与远端的编码时间，以及图片与特征的发送时间。

12.2.2 端侧编码的传输量与计算开销

编码器放在远端时，网络发送图片；放在终端时，网络发送特征。先沿图中的箭头确认两种方案分别传什么，再比较传输量和节省的远端计算时间。

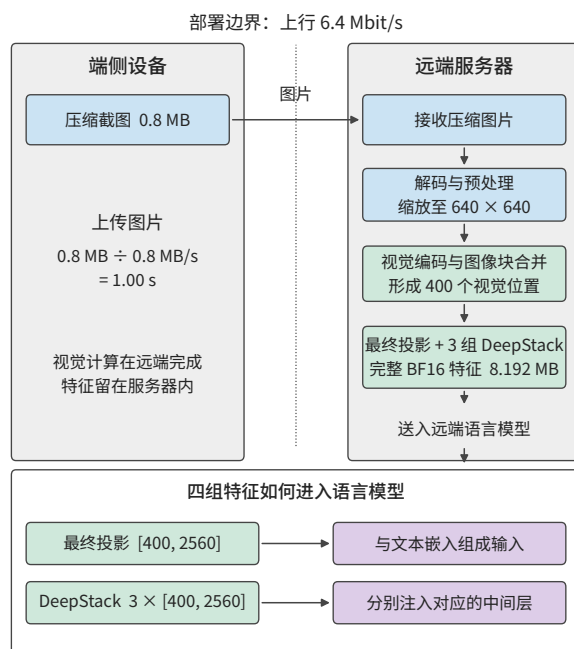


图 12-9：远端编码的执行路径。端侧上传 0.8 MB 压缩截图，远端依次完成解码与预处理、视觉编码和特征接入；最终投影与三组 DeepStack 特征均留在服务器内。6.4 Mbit/s 上行的图片发送时间为 1 秒，编码和排队时间另计。

例：上传视觉特征为何可能比上传原图更慢？ 沿用 0.8 MB 截图与 6.4 Mbit/s 上行。视觉编码采用 Qwen3-

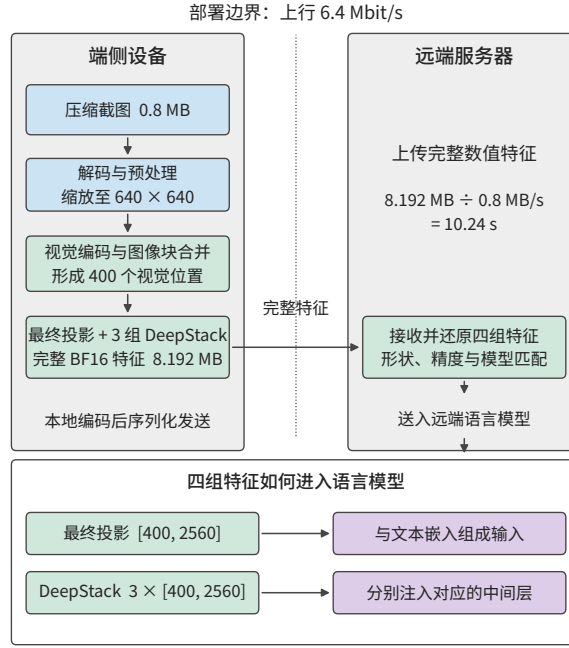


图 12-10：端侧编码的执行路径。预处理和视觉编码移到终端，跨网络的对象变为最终投影与三组 DeepStack 的完整 BF16 特征，共 8.192 MB；远端接收后仍按相同方式接入语言模型。相同上行的特征发送时间为 10.24 秒，本地编码、序列化和转换时间另计。图中数值采用下文的 Qwen3-VL-4B 固定配置。

VL-4B 的固定配置：预处理后 640×640 ，图像块合并后得到 400 个视觉位置，即用来表示图片内容的 400 个特征向量；这里的“位置”不是文本中的词元，也不是原图的单个像素。最终投影的每个特征向量有 2560 个分量，另有三组同宽 DeepStack 特征。DeepStack 将视觉编码器不同深度的特征送入语言模型不同层，因而要一起传送这些分支输出。BF16 是每个数占 2 字节的浮点格式。每个位置共需保存 $4 \times 2560 = 10240$ 个 BF16 数，完整输出为

$$S_E = 400 \times 10240 \times 2 = 8\,192\,000 \text{ bytes} \approx 7.8 \text{ MiB}.$$

四组特征分别输入语言模型的不同层。如果只传最终投影，就会漏掉另外三组特征。完整特征约 8.2 MB，是压缩图片的约十倍；在原上行中，发送时间从 1 秒增加到约 10.2 秒。⁵

将本地编码时间记为 $T_{E,l}$ ，远端编码时间记为 $T_{E,r}$ ，本地编码方案额外的序列化和转换时间记为 T_x 。两种方案的时间差为

$$\Delta T = T_{E,l} - T_{E,r} + T_x + \frac{S_E - S_I}{B_u}.$$

$\Delta T < 0$ 时端侧编码更快。式子中的前两项比较本地与远端的编码时间，最后一项比较图片与特征的发送时间。假设本地编码 0.12 秒、远端编码 0.04 秒、 $T_x = 0$ ，本地编码方案约需 10.36 秒，远端编码方案约需 1.04 秒。本地方案在计算和传输上都更慢：本地计算多 0.08 秒，特征发送多约 9.2 秒。

链路变快后，发送特征比发送图片多花的时间会减少。在 25 GB/s 上，图片与特征的发送时间相差约 0.30 ms。于是端侧编码要获益，只需在编码和排队上净省超过这 0.30 ms；在原慢上行上，则要省出超过 9 秒。所

⁵ 固定视觉形状与完整 EC / KV 计算、多模态阶段放置案例。固定视觉变体、DeepStack 及语言状态分别计量。

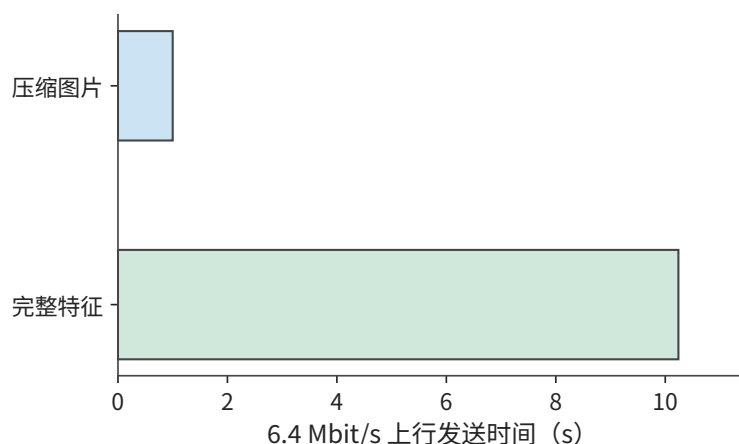


图 12-11：同一图片的完整视觉特征比压缩图片需要更多传输时间。压缩图片为 0.8 MB；Qwen3-VL-4B 在预处理后 640×640 输入上产生 400 个视觉位置，最终投影及三组 DeepStack 合计 [400,10240] BF16，共 8,192,000 bytes。上行速率为 6.4 Mbit/s，柱长表示发送时间；编码、排队和转换时间另计。两种方案处理同一张图片，完成相同的视觉理解任务。

以，决定编码器放在哪里时，慢链路上首先要比较传输量，快链路上则更需要比较编码和排队时间。

以上比较假定每张截图都重新编码。对多轮任务，还要检查图片是否重复出现，因为缓存可以进一步减少编码次数。相同图片再次出现时，缓存命中省掉一次编码；界面内容改变后则重新编码。在一组使用相同尺寸图片的 CPU 实验中，再次输入同一张图片时可以命中缓存，修改图片内容后则需要重新编码。⁶ 计算多轮任务的编码总时间时，可以用单次编码时间乘以缓存未命中的次数。这样就能把前面轮次留下的缓存也计入比较。

视觉特征只能省去视觉编码；如果希望连语言模型已经做过的计算也保留下来，就要保存注意力的上下文键和值 (KV)。编码缓存 (encoder cache, EC) 保存视觉编码器的输出；KV 则保存语言模型处理前缀时逐层产生的注意力状态。该配置每个语言位置需要 144 KiB 逻辑 KV，400 个视觉位置约为 56.3 MiB。图片、约 7.8 MiB 的视觉特征、约 56.3 MiB 的视觉位置 KV，分别对应不同的重算起点。缓存越接近模型的最终输出，再次使用时省下的计算越多，需要传送的数据也可能越大。下一节就比较传送一次状态要花多久，以及后续使用这些状态能省多少时间。

12.2.3 会话迁移的准备、复用与恢复

会话迁移将缓存和任务进度传到另一台设备，使它可以接着执行。目的端已有兼容权重时，需要传送的是编码缓存、语言 KV、任务记录和恢复信息。两端能够读取同一种状态格式后，就可以比较迁移前的准备时间与迁移后每轮省下的时间。

设迁移所需的准备时间为 M ，新设备每轮快 $\Delta t > 0$ ，剩余 N 轮。继续在原设备执行与迁移到新设备执行的时间差为

$$G(N) = N\Delta t - M.$$

$G(N) > 0$ 表示迁移获益。每增加一轮，收益增加 Δt ；准备 M 则把整条收益曲线向下移动。这解释了为什么同一迁移对短任务不划算，对长会话却值得。

⁶Qwen3-VL-8B、256×256 合成测试图像的完整 EC 约 2 MiB，PNG 约 3.4 KiB；8 次 TCP 传输前后的数据逐位一致，未运行完整语言请求。特征直传的计时没有包含发送前已完成的编码，不能直接据此判断端侧收益。来源：真实 Qwen3-VL-8B CPU 编码与传输。合成图像高度可压缩，运行环境为本机 TCP；不从中推断 WAN 收益。

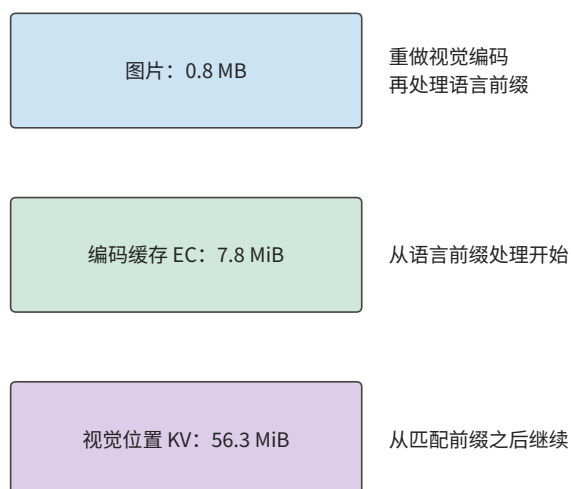


图 12-12：三种保存对象对应三个重算起点。EC 省去视觉编码；匹配模型权重、前缀与位置的 KV 进一步省去已处理前缀的语言计算。容量对应正文固定视觉配置。

例：剩余交互多少轮时，迁移状态才能缩短总时间？ 需搬运 64 MiB 状态，链路 80 Mbit/s，恢复 1 秒。准备为

$$M = \frac{64 \times 2^{20} \times 8}{80 \times 10^6} + 1 \approx 7.7 \text{ s.}$$

每轮快 0.4 秒时，10 轮净损失约 3.7 秒，20 轮净省约 0.3 秒，40 轮净省约 8.3 秒。用未舍入的 M 求 $N > M/0.4$ ，得到最少 20 轮。最少轮数说明何时开始回本，净收益大小则说明值得为这次切换承担多少额外成本。

图 12-13 中，迁移刚开始时净节省为负，之后每完成一轮就上升 0.4 秒。回本轮数对应曲线越过零线的位置。

例如，恢复再多花 1 秒，20 轮的净收益立即变成约负 0.7 秒，回本轮数增至 22。若预计后续还有 40 轮，新增这 1 秒之后仍可省约 7.3 秒。部署系统因此需要预测剩余会话长度，并为额外的传输和恢复开销留出时间。

曲线中的“恢复时间”包括新设备继续执行前所需的准备工作。除了装入缓存，它还要知道任务已经执行到哪一步。语言 KV 绑定模型、前缀和位置；截图 Agent 的进度由已经执行的操作决定。一次点击已经改变界面，即使其应答丢失，也应恢复到点击后的状态。操作编号和提交记录使目的端可以查到既有结果，从下一轮继续。保存这些记录可以避免重复执行操作，就像保存缓存可以避免重复计算。

连接预热、图编译和模型加载也可用 $N\Delta t - M$ 分析：将准备成本放入 M ，将后续每次减少的等待放入 Δt 。⁷ 这些准备是否值得，取决于后续调用省下的时间能否抵消一次性的准备时间。

⁷既有 survey 的第十二章编辑结论、框架覆盖记录、FlexSP / llm.npu 的固定正文与源码核对。按既有读取范围使用，不将历史原型当作当前部署支持。

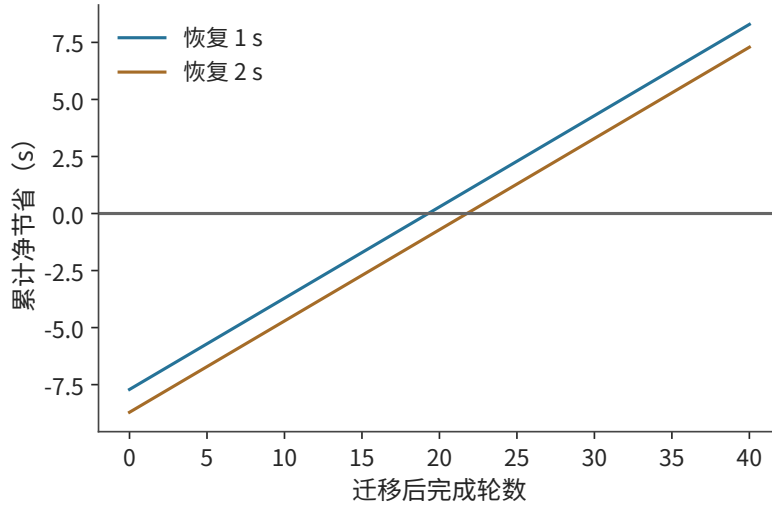


图 12-13：迁移 64 MiB 状态，链路 80 Mbit/s，迁移后每轮节省 0.4 秒。两条曲线分别取恢复时间 1 秒和 2 秒，净节省为 $N \times 0.4$ 减去传输和恢复时间。圆点标出首次获益的整数轮数；零线以上表示迁移更快，首次获益分别在第 20、22 轮。

12.2.4 模型内部切分的通信边界

会话迁移只在更换设备时传送一次状态。另一种分工方式是让多台设备一直共同执行模型，这时通信会在运行中反复发生。按阶段切分模型时，每个输入通常只需在相邻阶段之间传输一次；采用张量并行时，每层计算都要通信。链路从卡间互联换成局域网后，频繁同步会把微小的单次延迟累积成主要成本。

以 36 层 Qwen3-8B 的张量并行 $TP=2$ （每层矩阵分给两台设备）为例，考察每层注意力输出和前馈网络（FFN）输出的两次归约，归约将两台设备的局部结果相加，使后续计算得到完整输出。采用两阶段环算法。每次归约有两次启动，每个阶段启动为 α ，所以这些归约的单 token 启动时间为

$$T_{\text{start}} = 36 \times 2 \times 2\alpha = 144\alpha.$$

当 $\alpha = 2 \mu\text{s}$ 时，这部分约为 0.29 ms；当单阶段启动为 1 ms 时，就成为 144 ms。即使载荷很小，启动也必须等待；而下一层需要本层归约结果，自回归的下一个 token 又需要当前 token 的输出。由于这些步骤必须依次执行，每次通信的启动时间都会累加到单个 token 的完成时间中。⁸

下面两图依次展开一层的注意力输出与前馈输出，沿执行方向各经过一次归约后才能开始下一层。单次等待虽短，重复的次数却由模型结构决定。

流水线并行 PP 将层分成连续阶段，将通信集中到相邻阶段之间的激活传输。对单个请求，一个 token 仍需依次经过各个阶段；对多个独立请求，不同阶段可以同时处理不同请求。由此得到两种不同的收益：减少通信次数可以缩短单个请求的等待，让不同阶段同时处理请求可以提高总吞吐量。选择 TP 或 PP 时，先画出一个 token 的依赖，再安排多个请求占用各段资源，就能分别计算这两项变化。

确定模型分工后，需要传什么、传多少次，以及哪些计算要等待传输，都已经明确。接下来考察实际发送过程：数据准备好之后，还要看连接是否建立、窗口是否允许发送，以及确认是否及时返回。

⁸Qwen3-8B $TP=2$ 、 $PP=4$ 的逐操作通信量计算。正文只取 36 层的两次输出归约作为启动下界，完整计算还包括嵌入、阶段激活、logits 与 token 返回。

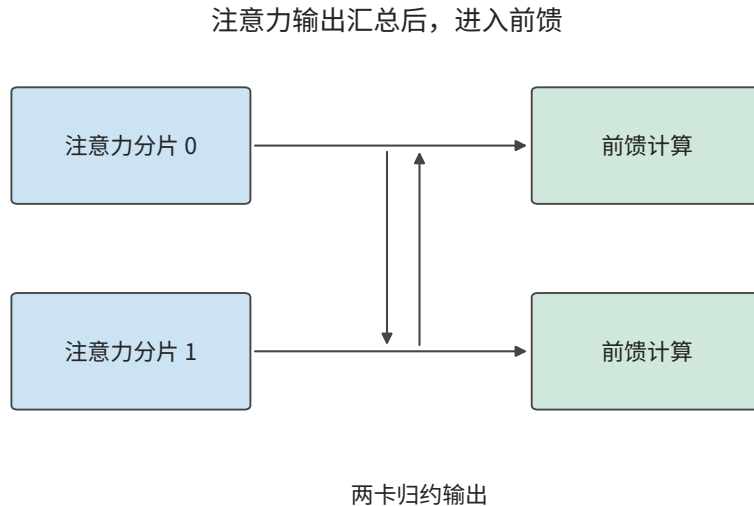


图 12-14：张量并行的一次注意力输出归约：两卡先分别计算，再交换并求和，取得完整输出后进入前馈。横向箭头表示执行顺序，中间纵向箭头表示两卡通信。

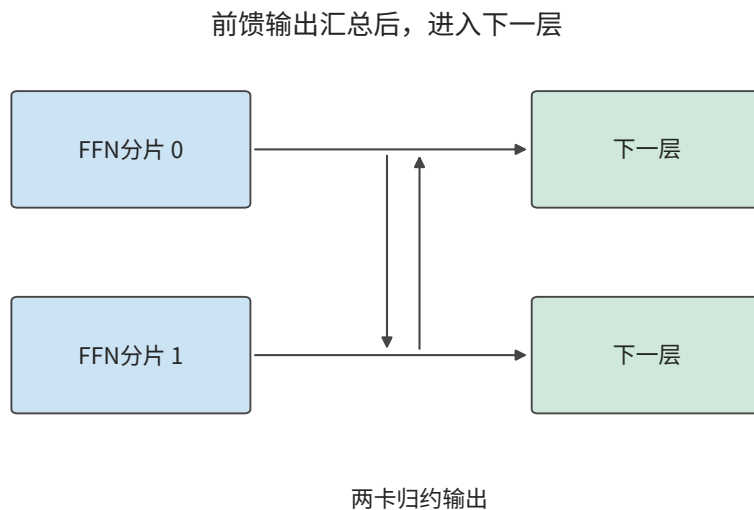


图 12-15：前馈网络也分别计算局部输出，再做两卡归约。完整前馈结果就绪后才能进入下一层，这一结构在 36 层中重复。

12.3 实际传输中的额外开销与等待

12.3.1 连接建立与复用

12.1 假设连接已经建立，链路以恒定速率传输数据。实际请求首先要建立连接和安全上下文，即通信双方为认证与加密维护的密钥、身份和会话状态；随后取得发送额度，最后等待接收端把数据交给应用。将这些等待加入前面的时间模型，就能分析实际执行过程：数据虽然已经准备好，却未必能立即计算或发送。

HTTP 是组织请求和响应的应用层协议。HTTP/1.1 和 HTTP/2 通常使用 TCP（提供可靠有序字节流的传输协议）与 TLS（建立认证与加密上下文的安全协议），HTTP/3 使用 QUIC（在 UDP 数据报上提供可靠多流传输和连接管理的协议）。UDP 向应用提供独立的数据报，不自行负责重传与排序。QUIC 将传输握手与 TLS 1.3 结合，减少分别建立传输连接和安全上下文所需的往返次数。会话恢复时，客户端还可以根据保存的安

全状态发送早期数据，即 0-RTT 数据。服务端接受后继续处理，拒绝后由客户端重发；服务端根据操作编号识别重试，并返回已保存的结果，从而避免重复执行同一操作。⁹

沿用 30 轮截图任务的 200 ms RTT，设建链额外需要两次往返。每轮重建会累计花 12 秒，只在第一轮建立则花 0.4 秒，省下 11.6 秒。连接复用之所以有效，是因为后续 29 轮共享了第一轮建立的状态。12.1 中的 105 秒采用已建立连接；加上首次建链后为 105.4 秒，每轮重建则为 117 秒。

连接建立是一次性开销，传输则每轮都要发生。请求频繁时，保留连接可以省去下一轮的建链时间；两次请求间隔很长时，空闲连接仍会占用内存等资源。关闭连接可以释放这些资源，但下次调用需要重新建链。与会话迁移一样，这里也要比较一次准备与后续多次使用的开销。

12.3.2 窗口、反馈与有效吞吐

复用连接省去了建链等待，连续发送数据还需要足够的发送窗口。连接建立以后，发送方也必须限制在途数据。拥塞窗口限制已经发送但尚未确认的数据量；接收流量控制根据接收端的缓冲空间，限制发送方还能发送多少数据。确认消息（ACK）告知发送方哪些数据已收到。ACK 到达后，发送方可以继续发送；接收端从缓冲区读取数据后，也会通知发送方新增的可用空间。达到发送上限时，即使链路空闲，发送方仍要等待这些通知。

在途量是已经发出、尚未得到确认的数据字节数。若允许在途量为 W ，往返时间为 R ，每个往返最多周转约一窗口的数据，因此

$$B_{\text{useful}} \leq \min\left(B_{\text{path}}, \frac{W}{R}\right).$$

式中 B_{path} 是路径可提供的载荷速率， B_{useful} 是实际有效吞吐。回到 20 Mbit/s、100 ms 的图片路径。要持续填满链路，需要约 250 KB 在途数据。若只有 64 KB 额度，速率上限为 $64000/0.1 = 640000$ bytes/s，约 5.1 Mbit/s，30 MB 仅发送就至少需约 47 秒。链路每秒本可以传输更多数据，发送方却要等下一批确认到来才能继续发送；扩大可用窗口才会提高这时的实际传输速率。

下面单独采用一个整批确认模型：接收端收齐 64 KB 后才确认，从这批发完到确认返回固定为 100 ms。因此完整周期包含 25.6 ms 发送和 100 ms 反馈，共 125.6 ms。图 12-16 将这段等待画出来。蓝色时段确实在发送，浅色时段链路虽然空闲，发送方却因为窗口已满而不能继续。物理带宽提高，只能缩短蓝色部分。

上图把窗口固定在 64 KB，以便看清停等过程。新连接通常从更小的窗口开始，随着确认到达逐步增加；短请求可能在窗口长大以前就已结束。设初始窗口为十个 1460-byte 段，每轮反馈后翻倍。前四轮累计可发 $14600(1+2+4+8) = 219000$ bytes，第五轮累计可达 452,600 bytes。约 355 KB 的语音请求因此要进入第五轮发送。对于 200 ms RTT，几轮反馈就是数百毫秒；短请求的大部分时间也可能花在等待确认上。12.5 将用这一机制解释一次语音性能观察。

窗口解释了发送为什么会暂停。完整时间还取决于究竟需要发送多少字节：网络上传送的除了图片，还有包头和确认。

例：协议头与 ACK 如何增加图片传输时间？ 在原 30 MB 上传、5 MB 下载上，取每包最多 1168 bytes 载荷，协议与网络头 60 bytes，每个数据包触发一个 92-byte ACK。合计约三万个数据包，头部与确认增加约 $30000 \times (60 + 92)/10^6 \approx 4.6$ MB，加上这些开销，实际传输量从 35 MB 增至约 39.6 MB。逐包计算发送、确认

⁹RFC 9000: QUIC 传输, §2、§7、§9、§13; RFC 9001: TLS; RFC 9114: HTTP/3; RFC 9221: 不可靠 Datagram。HTTP/3 是 HTTP 的传输映射，连接迁移、同时多路径与 FEC 分别判断。

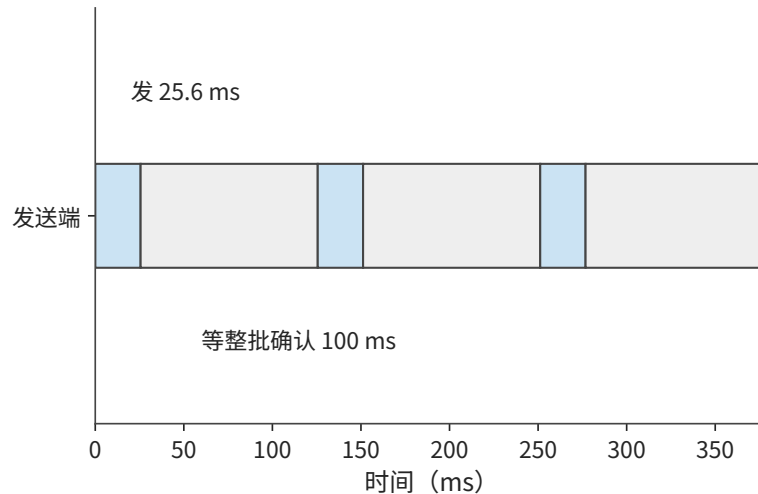


图 12-16: 固定窗口 64 KB, 链路 20 Mbit/s, 一批数据发送需 25.6 ms。为单独显示停等, 图设接收端收齐一批后统一确认, 从这批数据全部发出到收到整批确认再需 100 ms; 每批完整周期为 125.6 ms。

和后续发送的时刻, 完整请求约需 14.4 秒。¹⁰

与 12.8 秒相比, 多出来的约 1.6 秒来自模型中刚加入的工作与等待。单看实际传输的字节也仍不够: ACK 一面占用链路, 一面释放后续发送额度, 减少它会同时改变两项。

NewReno 是根据丢包与确认调整拥塞窗口的算法, CUBIC 在拥塞避免阶段按时间的立方函数调节窗口; 慢启动是连接初期随确认快速增加窗口的阶段。在相同的数据填充方式与 NewReno 配置下, 将立即 ACK 改为每两包确认或最多等待 10 ms, 实际传输的数据从约 39.6 MB 降至 38.2 MB, 完整请求却从约 14.52 秒增至 14.56 秒, 晚约 43 ms。¹¹ 这组对照揭示了反馈的双重作用: 少发确认减少传输时间, 确认晚到延长窗口等待。最终能否更快完成, 要比较省下的传输时间与增加的等待时间。

ACK 改变窗口继续发送的时刻, 拥塞控制则决定窗口本身如何变化。比较拥塞控制算法时, 应先找出请求经历了哪个阶段。相同路径的大图比较中, NewReno 与 CUBIC 完整响应均约为 14.5 秒; 窗口轨迹显示 CUBIC 一直在慢启动。¹² 本次请求在进入立方拥塞避免阶段之前就已结束, 因此观察到的是启动阶段的行为。要研究长流阶段, 就延长负载, 使窗口经历拥塞事件并进入相应控制过程。

12.3.3 多流传输与优先级

图片、音频和控制共用出口时, 存在两个可改变的顺序。发送调度决定谁先占用链路; 接收端的顺序要求决定哪些已到达的字节可以交给应用。前者决定链路先为谁发送数据, 后者决定接收端何时把数据交给应用。

先看资源分配。在一组共享链路的计算中, 先进先出 (FIFO) 按入队顺序发送, 它让图片第 5 秒完成、音频第 6 秒完成; 音频优先则让音频第 3 秒完成、图片第 6 秒完成。音频早了 3 秒, 图片晚了 1 秒, 原因是音频

¹⁰精确分包为上传 25,685 包、下载 4,281 包; 共 29,966 个数据包和同数 ACK, 线上 39,554,832 bytes, 完整响应 14.36339008 s。正文用约三万包、39.6 MB 和 14.4 s 说明量级。来源: 完整图片参考闭环。包长、立即 ACK、恢复与数值量化是明确输入, 该结果用于解释逐包反馈如何增加请求时间。

¹¹同一填充条件下, 立即 ACK 为 39,555,120 bytes / 14.52103724 s, 聚合 ACK 为 38,177,328 bytes / 14.564459796 s, 差约 1.38 MB 和 43.4 ms。14.36339008 s 属于另一无填充分支; 本节单因素比较使用这里具有相同填充条件的两组结果。来源: 立即 ACK 的 NewReno 对照、聚合 ACK 对照及 CUBIC 控制阶段记录。完整逐包日志保留在相应 JSON。

¹²同一填充条件下, 立即 ACK 为 39,555,120 bytes / 14.52103724 s, 聚合 ACK 为 38,177,328 bytes / 14.564459796 s, 差约 1.38 MB 和 43.4 ms。14.36339008 s 属于另一无填充分支; 本节单因素比较使用这里具有相同填充条件的两组结果。来源: 立即 ACK 的 NewReno 对照、聚合 ACK 对照及 CUBIC 控制阶段记录。完整逐包日志保留在相应 JSON。

把自身发送提前到原先由图片使用的时段。¹³

再看接收依赖。固定发送和丢包恢复时刻，音频自己的数据第 3 秒已经齐备，图片直到第 7 秒才齐备。这里把接收数据交给应用称为交付。整个连接共用一个接收顺序时，前方缺口会阻塞后续数据，这称为队头阻塞 (head-of-line blocking)。整体有序交付把音频也阻挡到第 7 秒；逐流交付在第 3 秒就交出音频，图片仍为第 7 秒。这里节省的 4 秒来自解除音频对图片接收进度的等待，发送的数据量和链路速率都没有改变。

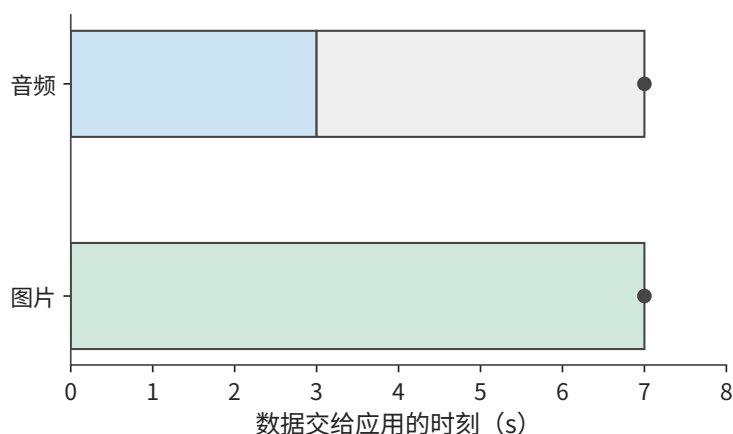


图 12-17：整体有序交付：音频第 3 s 已收齐，却因图片缺口继续等到第 7 s。灰色段表示数据已齐后的等待，圆点表示数据交给应用的时刻。

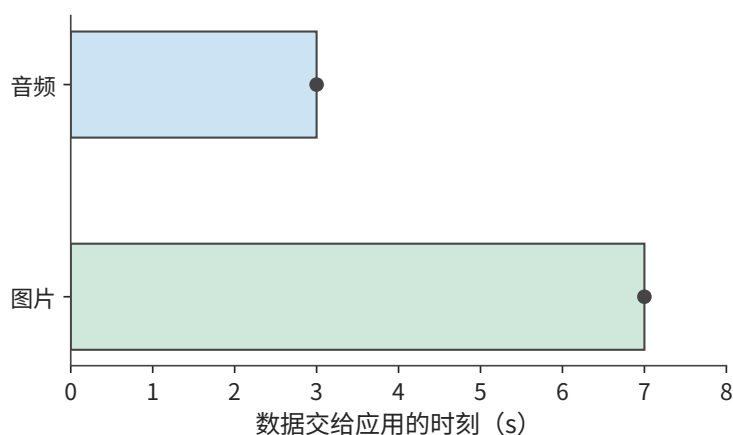


图 12-18：逐流有序交付：音频在第 3 s 收齐后立即交付，图片仍于第 7 s 交付。两图具有相同的发送、到达与恢复时刻；改变的是跨流等待依赖。

HTTP/2 将多个流入一个有序 TCP 字节流，前面的缺口会阻挡后续字节。QUIC 以流为单位保持有序，其他流已经连续到达的字节可独立交付。¹⁴ 这与音频播放的递推关系采用相同的分析方法：将一个覆盖全部数据的等待条件，分解成各流自己的等待条件，已经满足条件的流就能向应用交付数据。

这两种机制分别回答“何时发送”和“何时交给应用”。业务还要回答第三个问题：“数据交给应用时是否仍有价值？”音频的预定播放时段和截图版本给出了答案，下一节把这些期限与无线传输结合起来。¹⁵

¹³FIFO与音频优先；固定发送记录的整体有序与逐流交付。分别隔离调度和交付依赖。

¹⁴RFC 9000: QUIC 传输, §2、§7、§9、§13; RFC 9001: TLS; RFC 9114: HTTP/3; RFC 9221: 不可靠 Datagram。HTTP/3 是 HTTP 的传输映射，连接迁移、同时多路径与 FEC 分别判断。

¹⁵本地 TCP / HTTP3 传输与HTTP3 单连接多流。实现、负载与本地运行限制随原记录保留。

12.4 无线变化下的交互与恢复

12.4.1 共享空口与上下行争用

前一节已经区分了“何时发送”和“何时交给应用”。**空口 (air interface)** 是设备通过无线信号通信的接口。无线网络还改变了发送本身的代价：数据和反馈要争用同一段空口时间。在共享无线介质上，上行数据和下行反馈需要轮流占用空口。每次交换除了发送字节，还要等待获得信道使用权、等待帧间间隔并接收本跳确认。于是，应用字节数之外，还要计算交换次数。

端到端 ACK 告知远端发送方哪些传输已收到；介质访问控制 (MAC) 层组织单条无线链路的发送，MAC ACK 确认本跳无线帧。承载端到端 ACK 的无线帧，也会进行自己的 MAC 确认。因此，即使端到端 ACK 很短，也需要等待信道空闲，并完成本跳确认。

例：短 ACK 帧的无线接入与确认开销有多大？ 正交频分复用 (OFDM) 把数据分到多个相互正交的子载波上传送；PPDU 是物理层协议数据单元，包含物理层前导、头部和承载的数据，代表一次无线发送的完整物理层内容；SIFS 是连续交换之间的短帧间间隔。取无聚合的 OFDM 参考交换，数据速率 54 Mbit/s，MAC ACK 速率 6 Mbit/s。数据包交换包含 34 μ s 接入等待、208 μ s 数据帧、16 μ s SIFS 和 44 μ s MAC ACK，共 302 μ s。承载端到端 ACK 时，数据帧缩至 40 μ s，其余三项保持不变，共 134 μ s。¹⁶

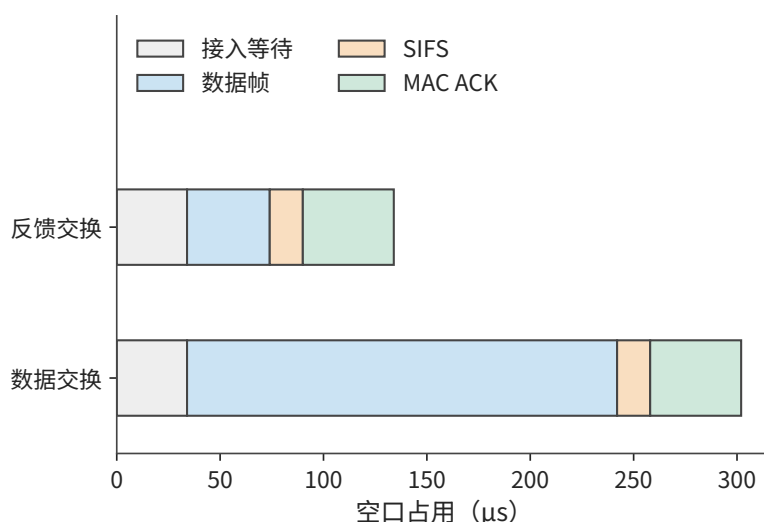


图 12-19：报文变短后，固定交换开销仍然存在，因此空口占用时间不会同比缩短。两种成功交换采用相同的 34 μ s 接入等待、16 μ s SIFS、44 μ s MAC ACK，区别只在数据 PPDU：208 μ s 或 40 μ s。条件为 OFDM54 / 6、无聚合、无重传的教学参考模型。

两种交换都包含 $34 + 16 + 44 = 94$ μ s 固定开销。数据帧从 208 μ s 缩到 40 μ s，缩短约八成，总交换却只缩短约 56%。固定项所占比例越大，继续缩短载荷的收益越小；把多个小报文合并发送、减少交换次数，因而能更直接地降低开销。

仍发送 30 MB 图片与 5 MB 成片，按该无线配置需要的数据交换和反馈交换各为 30,800 次，累计占用空口的时间约为

$$30800 \times (302 + 134) \mu\text{s} \approx 13.4 \text{ s.}$$

¹⁶ 共享空口图片计算。数据与 MAC ACK 分别采用 OFDM 54 / 6 Mbit/s，不使用帧聚合，接入等待时间由算例设定；MAC 层与端到端确认分别计量。

保持数据与恢复轨迹不变，将端到端 ACK 数减半，能释放约 $15400 \times 134 \mu s \approx 2.1$ 秒空口。上一节已经说明，反馈延迟又会增加窗口停等。因此，需要重新计算各包的发送与确认时刻：减少 ACK 腾出了空口时间，但等待确认又可能推迟后续发送。两者共同决定完整请求何时结束。

TACK 是减少无线确认开销的一种反馈机制。它将日常累计确认与需要即时反馈的事件分开：前者通过合并来减少交换次数，后者及时返回，使发送方尽快继续发送或重传丢失的数据。¹⁷ 这种分工同时减少确认报文的发送开销和等待反馈的时间。

12.4.2 业务时限、过期数据与取消

12.1 的播放器等待缺块，以保持全部内容；另一种播放器按固定时钟播放，晚到的音频块不再用于原定的播放时段。前者把网络延迟变成停顿，后者把延迟变成缺失声音。任务的质量要求决定采用哪一种播放规则。

例如，八块音频各长 20 ms，计划从第 0.4 秒开始连续播放，共 0.16 秒。让这些音频与大图共用链路时，FIFO 可使这些块最终全部到达，却全部错过各自预定的播放时刻；媒体优先则使八块都能按时到达。¹⁸ 两种结果有相同的最终字节数，按时可用的声音却分别为零与 0.16 秒。调度时统计按时播放的内容，才能反映用户实际听到了多少声音。

取消会让尚未播放的数据立即失去价值。播放器若排入三块 20 ms 音频，停止远端生成后本地仍可继续播出 60 ms。控制消息再用 100 ms 到达服务端，这段时间内还会产生新块。取消过程因此分成两条并行路径：本地清空待播缓冲，远端停止生成和发送后续音频。前者决定用户何时不再听到声音，后者决定何时停止消耗资源。

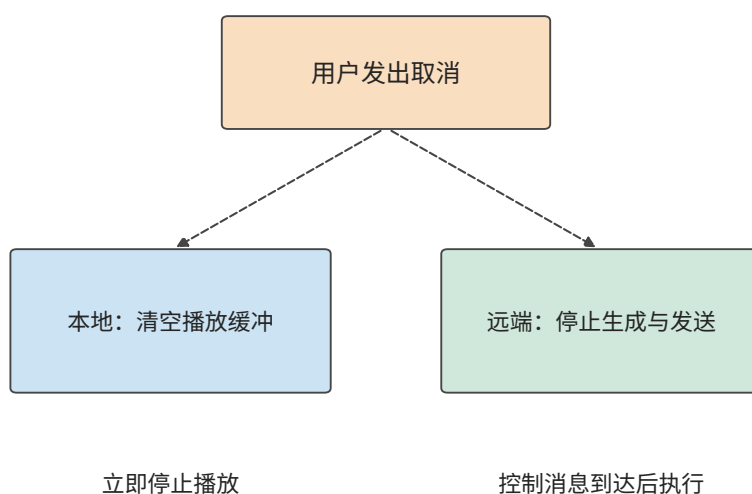


图 12-20：一次取消同时触发两条路径：本地清空待播音频，远端在控制消息到达后停止生成和发送。虚线表示控制流；本地停止播放与远端资源释放有各自的完成时刻。

截图 Agent 的动作则在提交后改变环境。应答丢失时，重试应查询原操作编号对应的结果；如果再次点击，就把通信重试变成第二次业务操作。保存操作结果后，网络恢复时就可以从原有进度继续，避免增加重复操作。

¹⁷ 计算机网络的新黄金时代（三）提供 TACK、Link Turbo 与代理部署的历史背景；NSDI 2026 survey 将 BLADE 列为备查。具体标准与实验条件优先于历史文章中的简化表述。

¹⁸ 媒体优先与聚合 ACK 的混合反馈计算及同目录 FIFO / 立即 ACK 对照。模型处理时间和预定播放时刻由算例设定，截图动作未完成的结果保留。

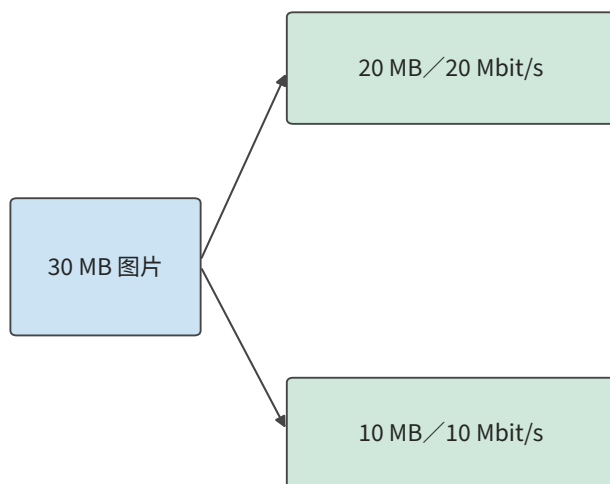
12.4.3 双路径选择、分流与故障切换

Wi-Fi 与蜂窝网络提供两条接入路径。分流把不同字节交给不同路径，目标是同时使用资源；复制把同一字节交给两条路径，目标是更早取得一份有效结果。这两种方法的数据量和完成条件不同，需要分别计算。

例：双路径如何分配图片数据以缩短传输时间？ 两条独立恒速路径分别为 20 和 10 Mbit/s，将比例 x 分给快路，其余分给慢路。两部分齐备才完成，因此

$$T_{\text{split}} = \max \left(\frac{xS}{B_1}, \frac{(1-x)S}{B_2} \right).$$

如果一条路径先结束，可以把另一条上的部分字节移给它，缩短较晚的结束时刻。最优比例因而使两条路径同时完成，即 $x = B_1/(B_1 + B_2)$ 。快路传 20 MB、慢路传 10 MB，各需 8 秒，比只用快路的 12 秒少 4 秒。



两路独立：均在 8 s 发完自己的部分

图 12-21：两条独立路径按带宽比例分配输入，分别发送 20 MB 和 10 MB，均需 8 s。任务等两部分都齐备后完成。

若两条路径汇入同一个 24 Mbit/s 出口，出口传完 30 MB 至少需 10 秒。两条接入路径可以同时发送，各需 8 秒，但所有数据都要经过同一个出口，在这里至少需 10 秒。此时瓶颈已经转移到共同出口。继续提高接入速率，只会让更多数据排队等待通过这个出口。

图 12-22 中，两条线在出口重新汇合。这个汇合点既解释了速度为什么不再相加，也引出了另一个问题：如果出口所在的设备故障，另一条接入能否继续传输？

如果目标是断连后仍能收到数据，就应把分流改成复制，并检查两份数据会在哪些地方同时失败。设共同端点失败概率为 q ；在它正常时，两条接入的失败相互独立，概率为 p_1, p_2 。失败包括端点失败，以及端点正常但两条接入均失败，两类互斥事件相加得

$$p_{\text{fail}} = q + (1 - q)p_1p_2.$$

取 $p_1 = 0.1, p_2 = 0.05$ ，接入同时失败为 0.5%；加入 $q = 0.02$ ，总体约为 2.5%。其中 2 个百分点直接来自共同端点。即使把两条接入做得完全可靠，总体失败概率仍为 2%；把服务副本放到不同故障域，才会降低这一

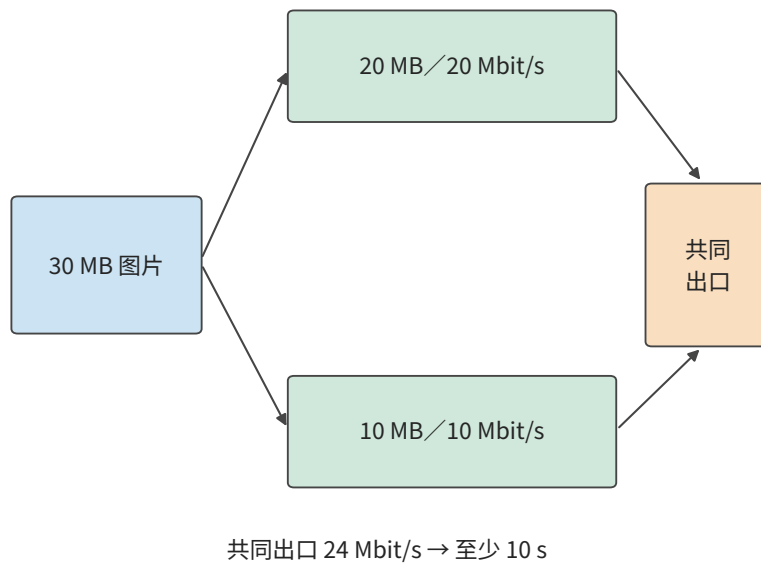


图 12-22: 30 MB 图片按 20 / 10 MB 分到 20 / 10 Mbit/s 两条独立接入, 各需 8 秒。两路再经过同一 24 Mbit/s 出口, 全部数据通过该出口至少需 10 秒。箭头表示数据路径, 不表示传播距离; 容量取恒定值。

项。

避开共同故障后, 复制还要支付额外带宽。以八块 640-byte 音频为例, 一份为 5120 bytes, 全量复制为 10,240 bytes。接收端用数据块编号去重, 先到的完整副本进入播放器, 后到副本被丢弃。多发送一份数据, 就能在两条路径中使用先到的结果; 如果两路共用同一个出口, 重复数据也会占用其他任务需要的带宽。¹⁹

多路径 TCP (MPTCP) 在一个逻辑连接中组织经过不同路径的 TCP 子流, 接入代理可在终端与普通服务器之间汇合这些路径。华为 Link Turbo 的多网络协同实践也利用不同接入的状态选择发送方式。²⁰ 代理的位置同时影响共享容量、传播与故障域, 因而成为部署模型中的一个计算和传输节点。

12.5 Queqiao 性能分析

12.5.1 初始测量与计时范围

本节使用作者归档的 Queqiao 传输系统实现开展分析。本节计时对象是一个固定音频请求从发出到收齐结果的路径。现在用 Queqiao 的语音传输案例检验前面的模型。固定音频文件约 355 KB, 链路 333 Mbit/s, 往返约 200 ms, 模型处理约需 30 ms; 计时从发起请求开始, 到收到结果结束。理想发送时间约为 8.5 ms, 加上传播与处理约为 240 ms。最初的直接调用却超过一秒, 而 Queqiao 约需三百毫秒。

这个差距有两种可能的解释。一种把优势归于传输机制本身; 另一种认为直接调用的大部分时间花在了连接准备、窗口增长和等待发送上。两者给出不同预测: 若连接准备与发送等待是主要原因, 充分调优直接路径后, 请求时间应当接近前面按数据量、计算时间和必要往返算出的最短时间; 若协议机制本身仍带来很大的优势, 差距会继续存在。

检验预测先要固定输入。早期测试轮换 146–405 KB 的八个文件, 再以最后一个文件大小描述整组中位数。

¹⁹双进程双 TCP 连接实验: 30 次正式尝试含共同端失败, 复制并非 first-wins, 不是实际无线测量。

²⁰计算机网络的新黄金时代 (三) 提供 TACK、Link Turbo 与代理部署的历史背景; NSDI 2026 survey 将 BLADE 列为备查。具体标准与实验条件优先于历史文章中的简化表述。

改为反复发送同一份 354,640-byte 文件，并交替运行两条路径之后，每次请求都发送相同数量的字节。²¹ 固定数据量后，再改变连接和发送配置，就可以比较它们如何影响请求时间。

12.5.2 调优基线与单因素比较

固定文件的两组结果如下。

连接条件	直接路径的请求中位数	Queqiao 的请求中位数	观察到的差距
新建连接	约 1.19 s	约 0.30 s	直接路径约为 3.9 倍
保持连接并调优	约 0.24 s	约 0.24 s	两者接近

新连接条件相差约 0.89 秒，调优保持连接后，两者均约为 0.24 秒。²² 原来的大部分差距随着基线配置改变而消失。这支持第二种解释：连接准备和发送方式在最初的差距中起了主要作用。即使使用同一种协议，配置不同，请求完成时间也可能相差很大。

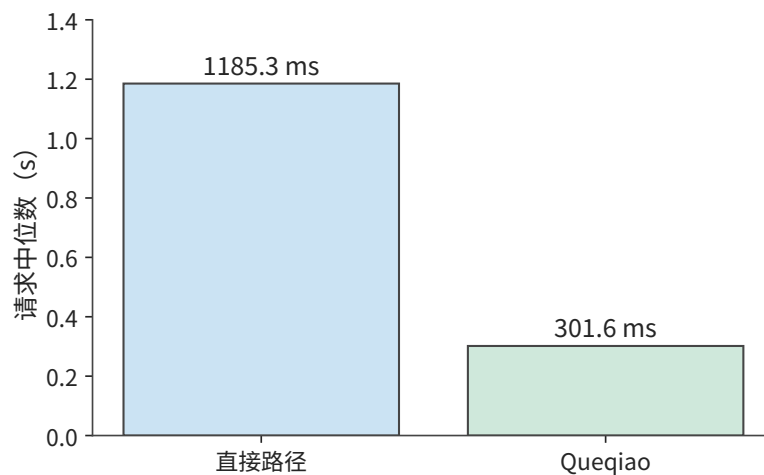


图 12-23：新建连接时，354640-byte 固定音频的直接路径与 Queqiao 请求中位数分别为 1185.3、301.6 ms。计时从发起请求到收齐结果，输入文件相同，两条路径交替运行。

用单因素实验区分连接复用、多流与确认策略的收益 上面的归档记录给出了整体配置对照。要继续区分各项设置的贡献，可以按下面的步骤设计单因素实验。从同一个客户端开始，只改变连接是否复用，测握手结束和首字节发送；再保持连接状态，只改变窗口设置，测在途量与 ACK 到达后的发送；最后固定这些设置，改变发送节奏，测链路空闲区间。每一步既记录总时间，也记录握手、发送和确认的时刻，就能说明时间究竟省在了哪里。

例如，如果等待窗口是主要原因，应当观察到这样的过程：发送一批数据后在途量达到上限，出口空闲，ACK 返回后立刻继续发送。若扩大窗口使这些空闲区间缩短，并使请求相应提前结束，就把窗口变化与总时间联系起来。只记录一个总时长，无法判断时间缩短是因为扩大了窗口，还是因为服务器处理变快。

实验的运行顺序也会影响比较结果。交替测试两条路径，可以让一组对比中的两次请求在相近时间运行，减小网络变化对比较的影响；同期 RTT 为约 197–205 ms，跨度约 8 ms。调优后的两条请求相差约 4 ms，小于同

²¹ 固定归档的路径记录与设计记录，来自提交 168ff4b。文件、调优设置和计时起止点均采用原记录中的定义。

²² 新建连接的直接 / Queqiao p50 为 1185.3 / 301.6 ms，调优保持连接为 240.9 / 236.5 ms；两个中位数之比分别约 3.93 / 1.019。文档另给逐轮配对对比中位数 3.96 / 1.03，统计量不同。正文将后一组写成均约 0.24 s；4.4 ms 的观测差不足以单独证明稳定优势。来源：Queqiao 同条件统计。区分两代负载、两个中位数之比与配对统计；分段中位数不可相加。

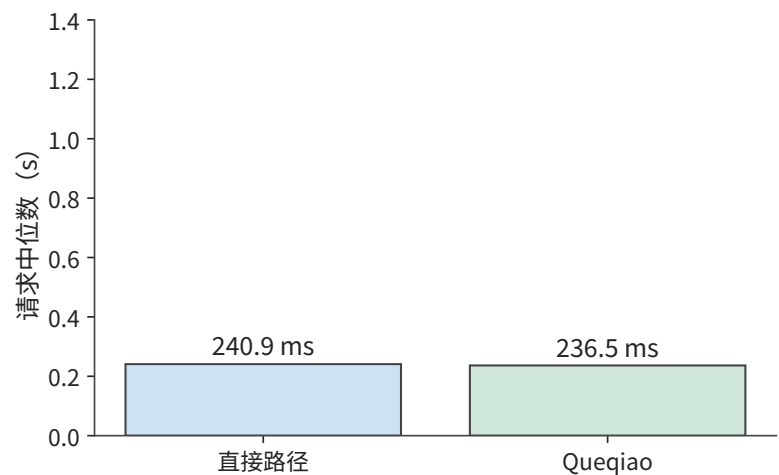


图 12-24：保持连接并调优后，同一固定音频的直接路径与 Queqiao 中位数分别为 240.9、236.5 ms。与前图采用相同纵轴，两条路径均接近数据发送、必要往返和处理所需的总时间。

单因素实验设计：每步只改变一个设置

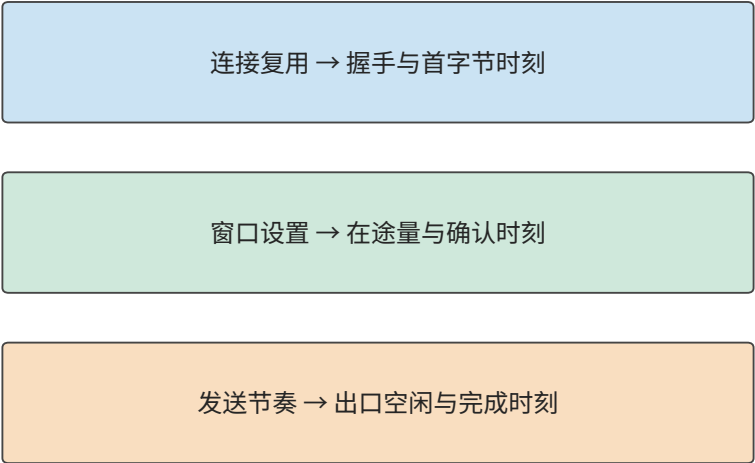


图 12-25：后续单因素实验的设计：保持相同文件、路径和其他设置，每次只改变一个因素，配对记录中间事件与总完成时间。此图列出实验步骤。

期 RTT 的波动幅度。进一步分辨小差异，应重复配对运行并研究差值分布；当前结果的主要信息是两条路径都已接近 240 ms。

12.5.3 实验结果与配置选择

调优对照方案后，下一步优化的重点也会改变。模型从 30 ms 加速到 3 ms，最多省 27 ms；将 RTT 从 200 ms 降至 20 ms，可省约 180 ms。只要附近执行新增的准备与计算少于这 180 ms，它就优于原远端路径。这与 12.1 的比较方法相同：先算出更换服务器可以少等多久，再扣除新增的计算和准备时间。

音频到达后，还需要由播放器连续播放。另一组采用静音输出设备回调的实验中，完整 PCM 最终全部收

到，回调却累计有约 0.36–1.44 秒取不到足够样本。²³ 请求结束只记录最后一批数据到达，播放器则在此之前就要持续读取音频。请求总时间与播放停顿时间描述的是两个不同的问题。

要查清停顿发生在哪里，需要记录每个音频块生成、到达、进入播放缓冲以及被播放回调读取的时刻。用 12.1 的递推式找到首次停顿，再检查此前各阶段的记录：音频块尚未生成，就检查模型处理；已经生成但未到达，就检查网络发送；已经到达却未被读取，就检查本地缓冲和调度。

这个案例展示了怎样从估算得出部署决定：先算出理想完成时间，再从执行记录中找出等待，用单因素实验解释原因，最后根据还能节省多少时间决定继续调优还是更换服务器。下面用同一方法完成截图 Agent 的部署。

12.6 从端边云分工到完整部署

12.6.1 地域、数据与服务位置

回到 12.1 的 30 轮截图任务。前十轮已完成，剩余二十轮要选择执行位置；从现在起，任务期限为 45 秒。无论选择哪种方案，此前已经花掉的时间都相同，比较的是剩余工作。原服务每轮 3.5 秒，继续执行需要 70 秒，因此需要换一种执行方案。

仍使用每轮 0.8 MB 截图、0.3 秒终端工作，原云路径上行 6.4 Mbit/s、RTT 0.2 秒。现在引入三个达到相同任务质量的模型服务：端侧每轮 2.9 秒，附近工作站 1.5 秒，云地域 0.8 秒。各服务的初始化和上下文准备合计列在“一次准备”中。先采用无排队、无故障、控制回复发送时间为零的教学条件。

执行设备	一次准备	每轮模型	上行	每轮往返传播	20 轮基础成本
端侧	0 s	2.9 s	无需上传	0 s	0.060 成本单位
附近工作站	3 s	1.5 s	80 Mbit/s	0.02 s	0.080 成本单位
云地域	1 s	0.8 s	6.4 Mbit/s	0.20 s	0.050 成本单位

基础成本按每轮 0.003、0.004、0.0025 计收，准备成本包含其中。云路径另按每 MB 上传 0.001 收费，其他路径传输包含在基础成本。这样，轮数决定执行成本，字节数决定额外传输成本。

将云端进一步分为不同地域后，仍然需要比较这些时间与成本。采用 Regionless（由平台为请求选择云地域，而非应用固定绑定地域）服务时，应用指定任务、质量要求和完成期限，由平台选择部署地域。²⁴ 平台仍需比较各方案的实际开销：服务器距离影响传播时间，缓存影响重新计算的工作量，跨地域传输影响数据传输成本。下面先确定能否按时，再比较成本。

12.6.2 时限、成本与恢复的联合比较

例：状态迁移与剩余交互轮数如何改变端边云部署选择？ 剩余二十轮的时间为

²³8 次完整接收测试中，收到的 PCM 与播放回调读取的 PCM 均逐位一致，播放回调累计有 360–1440 ms 无法取得足够音频样本；设备输出静音，无声学起播测量。3 次取消早于首个回调，旧验收要求回调非空而报错；根据已收到的字节、连接关闭事件和远端 EOF，可以判断这些请求在首次播放前就已取消；统计时保留了零回调记录。完整四轮数据见配套证据说明。来源：显式物理接口绑定的音频实验，固定提交 496ca627，基线含 socket 绑定适配。与历史 ASR 记录分开，静音设备回调不等于声学测量。

²⁴计算机网络的新黄金时代（二）的云际网络与 Regionless 讨论。本章地域方案为独立教学预算，没有引用实际市场价格。

$$T_{\text{local}} = 20(0.3 + 2.9) = 64 \text{ s},$$

$$T_{\text{near}} = 3 + 20(0.3 + 1.5 + 0.02 + 6.4/80) = 41 \text{ s},$$

$$T_{\text{cloud}} = 1 + 20(0.3 + 0.8 + 0.20 + 6.4/6.4) = 47 \text{ s}.$$

云端每轮比附近工作站少算 0.7 秒，二十轮省 14 秒，准备再省 2 秒；但上传每轮多 0.92 秒，传播多 0.18 秒，二十轮多等 22 秒。合计下来，云端反而慢 6 秒。把时间差拆开，便能看到更快的模型怎样因每轮通信而失去优势。

图 12-26 至图 12-28 将各项差额放回完整任务。云端的绿色计算段更短，蓝色上传段却更长；应比较整条横条的终点，而不是只看模型计算那一段。

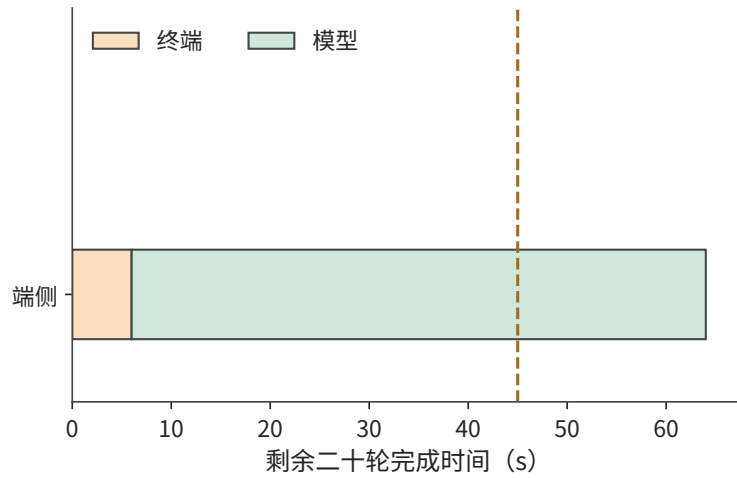


图 12-26：端侧：终端工作 6 s，模型计算 58 s，共 64 s。三图共用 45 s 期限线和同一横轴，颜色分别汇总二十轮同类工作的耗时。

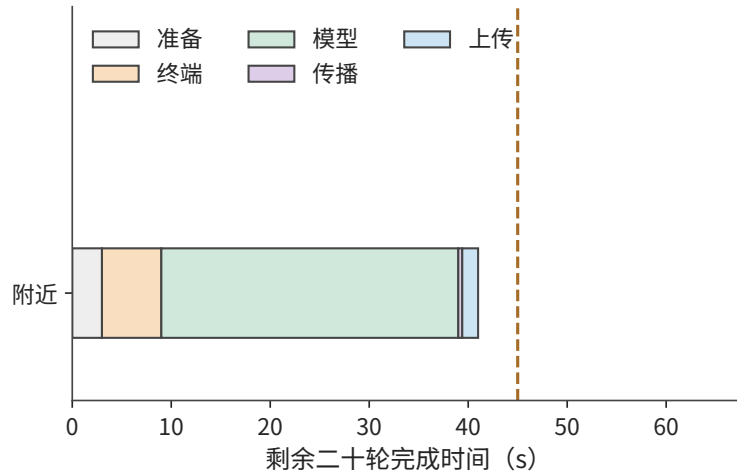


图 12-27：附近工作站：准备 3 s，二十轮终端 6 s、模型 30 s、传播 0.4 s、上传 1.6 s，共 41 s，满足 45 s 期限。

45 秒期限内只有附近工作站可行。端侧成本 0.060 较低，但需 64 秒；云端总成本为 $0.050 + 20 \times 0.8 \times 0.001 = 0.066$ ，但需 47 秒。因此，在能按时完成的方案中，成本最低的是附近工作站方案，为 0.080。

只改变云上行 b ，单位 Mbit/s，其他参数保持不变，云端完成时间为

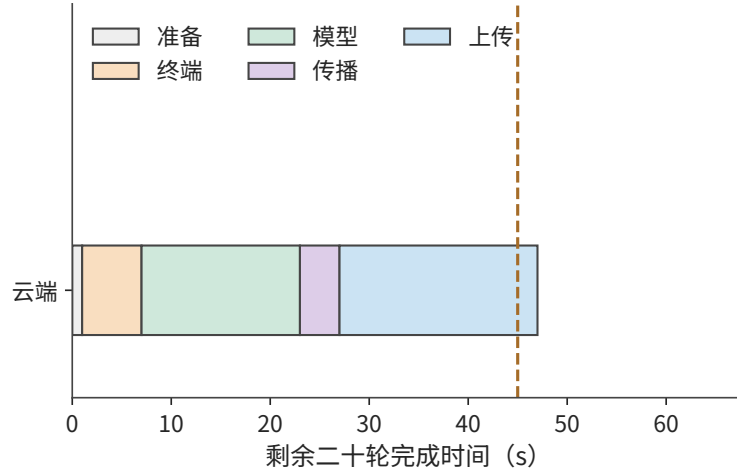


图 12-28: 云端: 准备 1 s, 二十轮终端 6 s、模型 16 s、传播 4 s、上传 20 s, 共 47 s。计算更快而传输更久, 总时间超过期限。

$$T_{\text{cloud}}(b) = 27 + \frac{128}{b} \text{ s.}$$

27 秒来自一次准备和二十轮非上传工作, 128 Mbit 是二十张截图的总上传量。满足期限需要 $b \geq 128/18 \approx 7.1$ Mbit/s; 比附近的 41 秒更快需要 $b > 128/14 \approx 9.1$ Mbit/s。上行 8 Mbit/s 时, 云端任务需 43 秒, 已经按时且更便宜; 10 Mbit/s 时约 39.8 秒, 才同时更快。

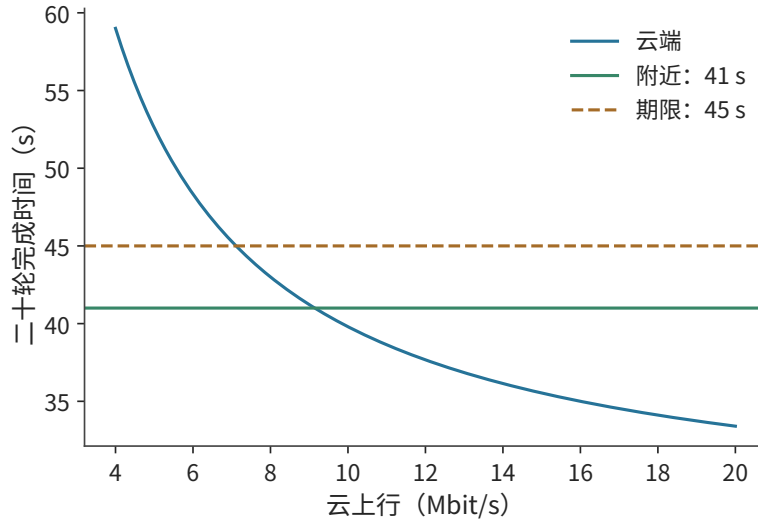


图 12-29: 云路径上行改变完整任务的部署选择。剩余 20 轮每轮上传 0.8 MB, 云端准备 1 秒, 每轮其余工作合计 1.3 秒, 因此总时间为 $27+128/b$ 秒; 附近工作站为 41 秒, 期限为 45 秒。本例保持模型质量、处理时间、价格和其他网络参数不变, 忽略排队与故障。约 7.1 Mbit/s 起云方案可满足期限, 约 9.1 Mbit/s 以上才比附近工作站更快。

接近期限时, 还要计算剩余的时间。上行 8 Mbit/s 时, 任务可以提前 2 秒完成; 降到 7 Mbit/s, 时间约为 45.3 秒, 已经超期。若要求 99% 的任务按时, 就需要分析任务完成时间的概率分布。由于每轮速率 b_i 可以不同, 包含上传在内的总时间应写为

$$T = 27 + \sum_{i=1}^{20} \frac{6.4}{b_i}.$$

每轮速率在其上传期间恒定时，十轮 6 Mbit/s、十轮 10 Mbit/s 的平均速率为 8 Mbit/s，但任务约需 44.1 秒，比全程 8 Mbit/s 的 43 秒更慢。慢速增加的等待大于快速省下的等待，原因是发送时间与速率成反比。平台应根据整个任务的上传时间分布，判断是否满足 $P(T \leq 45 \text{ s}) \geq 0.99$ ，还要考虑网络持续变慢时，相邻多轮会一起受到影响。

前面的部署选择主要受截图上传时间影响。换成输入很小、前缀重复很多的文本任务，优化重点会转向计算复用。保留会话状态还可以减少重复计算。考虑一条文本 Agent 会话：逐轮重送前缀合计约 66 KB，保留状态后只送约 10 KB，回复均约 2.5 KB。在 333 Mbit/s 上，少传约 56 KB 只省 1.3 ms。²⁵ 与截图的 16 MB 上传相比，这里的主要收益来自少做前缀计算。

同一会话的矩阵计算量从约 3.0×10^{14} 降至 6.0×10^{13} FLOPs。按每秒 10^{14} FLOPs 的计算速率、每 GPU 小时 1.2 成本单位计费，可以省下约 0.00079 成本单位。保存约 465 MB 状态，按每 GB 小时 0.05 计费，每秒成本约 $0.465 \times 0.05 / 3600 \approx 6.5 \times 10^{-6}$ 。两者相除，保存约 120 秒就花完本次省下的计算成本。这种盈亏平衡的计算方法与前面的迁移时间比较相同：复用收益必须覆盖保存或搬运状态的成本。

保存状态既能减少正常执行中的重复计算，也能减少故障后的重做。下面回到附近工作站上的截图任务，比较同一次断连如何因进度保存方式不同而产生不同开销。附近工作站完成一轮任务需要 1.9 秒。设第十轮已执行完，但确认尚未返回时连接中断，恢复连接需要 1 秒。若前九轮进度已经提交，只重做未确认的第十轮，额外花 2.9 秒，任务由 41 秒增至 43.9 秒；若十轮都要重做，额外花 20 秒，任务变为 61 秒。仍然使用同一台工作站，能否保留进度却决定了任务能否按时完成。

图 12-30 标出了两种恢复方式真正不同的地方：相同的断连事件之后，一种只重复最后一轮，另一种重复此前十轮。恢复开销有多大，首先取决于哪些结果已经可靠保存。

上图比较了一次已发生的断连。若要估计大量任务的平均成本，还要把失败发生的概率计入：重试也会增加成本。每次尝试独立、成功率为 p ，失败后全部重试且每次收费 C ，期望成本满足 $E = C + (1 - p)E$ ，所以 $E = C/p$ 。99% 成功率对应约 $1.01C$ ，80% 对应 $1.25C$ 。保存进度可以把整项重试变成局部重做，直接降低每次失败追加的时间和成本。

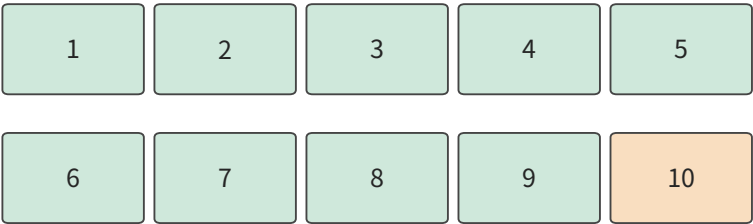
12.6.3 何时需要更换部署方案

本例应把剩余二十轮放到附近工作站：41 秒完成，成本 0.080。与继续使用原服务的 70 秒相比，省 29 秒。终端继续采集并执行操作，工作站负责模型判断，每轮只传输截图和控制结果。

这个决定来自对三种部署方案的逐项比较。端侧先因 64 秒超期被排除；云端虽然模型更快、成本更低，但原上行使它需要 47 秒；附近工作站虽然执行模型较慢，但通信时间较短，整个任务反而更快完成。前面的压缩、特征与状态分析则给出了进一步改进的方法：压缩截图可以减少上传时间，上传完整特征会增加传输时间，保留已提交的进度则可以减少恢复后的重复工作。

²⁵ 同一模型、token 序列与轮次的重放，入向原值 66,207 / 10,193 bytes，出向均为 2,527 bytes，非三位置独立性能实测。工作量除以设定的计算速率仅为时间下界。状态 465,371,136 bytes，成本回本约 121.8 s；远端保留会话状态与本地执行两种方案的总成本相等时，对应的出网单价约为 84.17 / GB，设定的费率 0.08 / GB，正文推导了保留状态多长时间可以节省成本。来源：固定 Agent 轨迹的地域放置与 12 周资源交付期限。字节来自原轨迹，价格与计算速率是算例设定值，保留状态的时长与出网单价分别对应两种不同的成本比较。

已执行十轮；恢复连接需 1 s

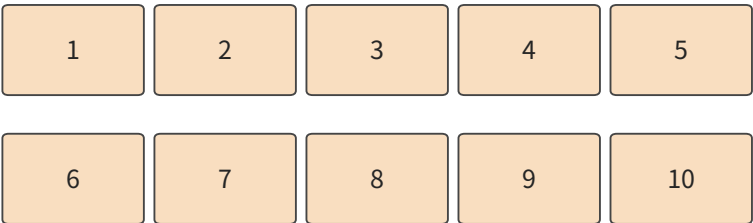


保留前九轮，只重做第十轮

额外 $1 + 1.9 = 2.9\text{ s}$

图 12-30：保留前九轮提交记录（绿色），只重做未确认的第十轮（橙色）。每轮 1.9 s，恢复连接 1 s，增加 2.9 s，总时间为 43.9 s。此例采用可安全重放的操作；已提交的外部操作则先查询其结果。

已执行十轮；恢复连接需 1 s



进度丢失，重做十轮

额外 $1 + 10 \times 1.9 = 20\text{ s}$

图 12-31：丢失十轮进度时，恢复连接后重做十轮，额外 $1 + 10 \times 1.9 = 20\text{ s}$ ，总任务从 41 s 延至 61 s。

条件变化	结果	决策
原云上行 6.4 Mbit/s	云端 47 s，附近 41 s	使用附近工作站
云上行升至 8 Mbit/s	云端 43 s、成本 0.066	云端方案可以按时完成，成本也更低
云上行升至 10 Mbit/s	云端约 39.8 s	云端同时更快、更便宜
附近发生一次断连，保留前九轮进度并重做第十轮	43.9 s	仍在期限内
同次断连丢失十轮进度	61 s	需要改进进度保留机制

任务执行期间更换设备，还需要迁移会话状态。12.2 的 64 MiB 会话需要约 7.7 秒准备，每轮快 0.4 秒，20 轮只净省约 0.3 秒。若带宽改善只持续十轮就恢复原状，这次迁移会净损失约 3.7 秒。因此，平台应将预测的持

续收益与切换成本比较，只有预计节省的时间足以抵消迁移准备，并能容纳额外开销时，才迁移状态。在一段时间内继续使用同一台设备，可以避免在临界带宽附近反复迁移状态。

第 1 章通过容量、运算和读取时间判断设备能否承载模型；本章进一步分析通信、执行顺序和恢复过程，判断它能否按时完成任务。选择部署方案时，需要依次回答：任务包含哪些工作，哪些步骤必须等待，实际执行还会增加哪些开销，以及条件变化后哪种方案更合适。

12.6.4 资源条件变化后的系统调整

将前文的硬件与执行方式比较用于完整任务。设一次固定轨迹中，模型执行合计 8 秒，工具与网络等不可重叠部分合计 2 秒。即使模型整体加速 10 倍，任务也只能从 10 秒降至 2.8 秒，约快 3.57 倍；²⁶模型耗时趋近于零时，下界仍是 2 秒。

要继续缩短剩余两秒，可以改变系统组织。把执行移到本地可减少网络往返，保留环境可减少创建等待；相应增加的本地设备与驻留空间，则进入新的成本预算。前面的部署算例已经给出了比较方法：先求满足质量与期限的方案，再比较每个完成任务的总成本。

V4 到 V4.1 展示了模型侧的同类调整。跨层共享减少全局 KV 的重复保存，使模型可以用更细的历史粒度；CED 减少输入阶段反复经过解码器的工作，缓存恢复则缩短多轮会话重新开始的路径。²⁷这些改变分别作用于容量、输入计算和恢复等待，可以沿同一任务逐项计入预算。

从片上存储到端边云，设计选择始终依赖对完整执行过程的了解。注意力的归约关系使中间矩阵可以按块处理，重复的模型结构使执行计划可以复用，前缀与分支关系使上下文可以共享，工具等待则为状态换出和环境准备提供了时间。每一项改进都把模型或任务的信息用于其他层次，重新安排数据保存位置、计算划分和交接方式。先保持任务不变，计算直接收益，再根据新约束调整模型与部署，就能说明性能提升来自哪里，以及下一步应改变什么。

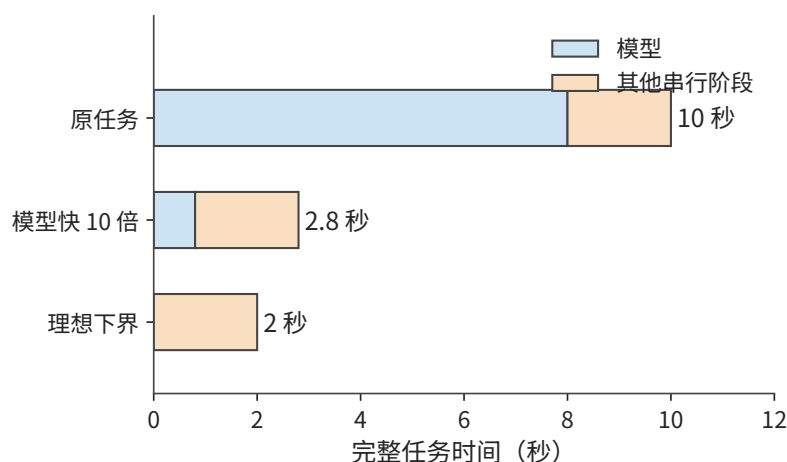


图 12-32 固定任务轨迹中的加速上限。模型从 8 秒缩短至 0.8 秒，其他串行阶段保持 2 秒；第三行表示模型时间趋近于零的理想下界。

²⁶本章新增条件比较的逐项复算与计算程序。

²⁷DeepSeek V4.1 官方技术报告，第 1、2、3 节与第 6 节；跨章会话的固定条件与复算。架构、格式和部署机制按报告区分，教学时间假设不作实测。

练习

练习按计算、改变条件和解释机制递进。12-2、12-3、12-7 为核心练习。未另说明的数值沿用正文教学条件。

12-1 图片精修的传输与计算预算〔延伸, ★〕。

- (a) 分别取上行 10、20、100 Mbit/s, 计算原方案、处理十倍加速和输入减半方案的完成时间。压缩新增编解码仍为 0.15 秒。
- (b) 求压缩恰好回本的上行速率。若只有 0.3 秒处理中的 0.2 秒可以加速, 十倍加速后的完整时间是多少?
- (c) 把图像分为三个可独立处理的块, 画出上传、处理与回传的重叠。标出使整项完成时间无法继续下降的工作。

12-2 远端执行节省与增加的时间〔核心, ★〕。

- (a) 复算 0.8 MB 降至 0.2 MB、新增编码 30 ms 时, 30 轮截图任务的节省。
- (b) 压缩后需要 38 轮时, 任务是否更快? 求仍有收益的最大整数轮数。
- (c) 将每轮 RTT 增加 100 ms, 分别计算原图 30 轮和压缩图 38 轮的总时间。解释为什么同一个 RTT 变化会产生不同的任务耗时增量。
- (d) 为截图版本与操作编号设计记录字段, 说明怎样识别压缩导致的纠错轮次与旧状态导致的无效轮次。

12-3 AI 助手的跨设备分工〔核心, ★〕。

- (a) 用 [400, 10240] BF16、0.8 MB 图片和 6.4 Mbit/s 上行, 求本地编码 0.12 秒与远端编码 0.04 秒两种方案的时间差。
- (b) 改为 25 GB/s, 求使端侧编码有利所需的计算与排队净节省。
- (c) 搬运 64 MiB, 速率 80 Mbit/s, 恢复 1 秒, 每轮快 0.4 秒, 求 10、20、40 轮的净收益。恢复再增加 1 秒, 最少回本轮数如何改变?
- (d) 分别说明保存图片、视觉特征和语言 KV 后, 从何处开始重算; 说明已提交的操作结果怎样进入恢复过程。

12-4 连接复用与多流传输〔延伸, ★〕。

- (a) RTT 为 200 ms, 任务共 30 轮, 每次建链需额外两次往返。计算每轮新建连接与全程复用连接的时间差。
- (b) 复算图片传输中包头与 ACK 的总字节数, 解释为什么实际传输的字节更少, 请求完成时间反而可能更长。
- (c) 比较发送优先级与图 12-18 的逐流交付: 分别指出资源使用时段和等待依赖发生了什么变化。
- (d) 设计仅改变连接复用的匹配实验, 列出起止事件及用于解释时间变化的中间事件。

12-5 ACK 频率与空口效率〔延伸, ★〕。

- (a) 数据交换 302 μ s、反馈交换 134 μ s, 数据交换 30,800 次, 求 ACK 减半释放的空口时间。
- (b) 画出发送方因等待聚合 ACK 而暂停发送的时间线, 解释释放空口与提前交付之间的关系。
- (c) PCM 播放所需速率为 256 kbit/s、实际接收速率为 130 kbit/s、初始缓冲 60 ms, 求耗尽时刻。保持接收速率和 60 ms 初始音频时长不变, 将采样率从 16 kHz 改为 24 kHz, 求新的耗尽时刻。

12-6 双路径的分流与备份〔延伸, ★〕。

- (a) 30 MB 经 20 与 10 Mbit/s 两路分流, 推导使两路同时完成的比例与时间。
- (b) 加入 24 Mbit/s 共同出口, 求新的资源下界, 并指出增加哪一段容量能够降低它。
- (c) 两条接入路径失败概率为 0.1、0.05, 共同端点为 0.02, 求复制后失败概率; 两条接入路径完全可靠后, 该概率是多少?
- (d) 八块 640-byte 音频全量复制, 求新增字节, 并说明接收端怎样避免重复播放。

12-7 Queqiao 的单因素比较〔核心, ★★〕。

- (a) 计算新建与调优保持连接两组中位数之比，说明“主要差距来自连接准备与发送等待”这一解释给出了什么预测。
- (b) 用约 355 KB、333 Mbit/s、200 ms RTT、30 ms 处理求理想时间，指出最初一秒级结果中待解释的大致有多少等待时间。
- (c) 设计窗口大小的单因素实验，说明如何交替运行，以及何种在途量、ACK 和发送轨迹支持窗口解释。
- (d) 使用配套音频记录，画出一个块从生成到播放的各个时刻，说明如何区分生成不及时、传输延迟与本地调度等待。

12-8 跨地域计算与数据成本（延伸，★★）。

- (a) 复算剩余二十轮三种部署方案的时间、成本和两项带宽要求。
- (b) 改为剩余十轮、期限 23 秒，选择按时且成本最低的方案；再把云上传单价加倍，重新选择。
- (c) 二十轮中十轮上行 6 Mbit/s、十轮 10 Mbit/s，求云端的总时间，与恒定 8 Mbit/s 比较并解释差额。
- (d) 对十轮任务，设第八轮已执行完时断连，恢复 2 秒。分别计算保留前七轮、重做第八轮，以及丢失全部八轮两种情形的剩余与总时间。这次断连发生在远程服务路径，端侧继续正常执行；恢复成本包含在基础费率中。恢复期间连接与服务状态保留，初始化只做一次；成本按实际执行轮次及发送字节计收，按原上传单价重新判断最低成本的可行方案。

本章小结

完整交互应从三个层次分析。先根据数据量、计算量和传播距离求出各步骤所需时间，再以执行依赖确定哪些时间相加、哪些可以重叠，最后加入连接、窗口、竞争和恢复来解释实际等待。图片的 12.8 秒完成时间、语音的逐块递推和 Agent 的逐轮累积，分别展示了这三个层次怎样配合。

模型分工改变了跨设备传输的数据形式和次数。把计算移到端侧，可能把紧凑图片换成更大的特征；把模型拆到多个设备，可能把一次传输换成逐层同步。比较时先扣除两种方案中相同的耗时，再比较省下的计算时间与增加的通信时间，便能找到收益来自哪里。

复用把一次准备分摊到后续调用。迁移、连接预热、图编译与状态保存，都可以根据准备成本和每次省下的时间，求出至少需要使用多少次才划算。预期节省的时间或成本，还需要留出一部分来承担波动和恢复开销。

设计实验时，也可以沿用同一套时间模型。固定输入和计时的起止点，记录各阶段的等待，再逐项改变设置，检查总时间的变化能否由具体阶段的变化解释。Queqiao 的基线修正展示了这一过程：调优消除大部分额外等待后，进一步改善性能就需要缩短传播时间。

最终选择先满足质量和期限，再比较成本。即使平均计算速度和传输速率足够，任务仍可能因慢速轮次或丢失进度超期；按时完成的概率与恢复后的重做量，因而是部署模型的组成部分。

再看第一章提出的模型应用，开发者通过上下文表达任务，执行系统据此安排计算、保存状态并传输数据。现在，保留一段历史、增加一个候选分支、提前准备工具环境或迁移一次会话，都有了可以追踪的资源代价。理解这些代价，也就能够识别哪些通用执行开销可以利用应用信息消除，并根据任务质量、完成时间和总成本选择实现方式。这是学习 AI Infra 带来的系统设计能力。